https://doi.org/10.1007/s11704-026-51539-5

RESEARCH ARTICLE

An improved quantum software challenges classification approach using transfer learning and explainable AI

Nek Dil KHAN[1], Javed Ali KHAN[2]✉, Mobashir HUSAIN[3], Muhammad Sohail KHAN[4], Arif Ali KHAN[5], Muhammad Azeem AKBAR[6], Shahid HUSSAIN[7]

1. Faculty of Information Technology, Beijing University of Technology, Beijing 100081, China
2. Department of Computer Science, University of Hertfordshire, Hatfield, AL10 9AB, UK
3. Department of Computer Science & IT, Sarhad University of Science and Information Technology, Peshawar 25000, Pakistan
4. Department of Computer Software Engineering, University of Engineering and Technology Mardan, Mardan 23200, Pakistan
5. M3S Empirical Software Engineering Research Unit, University of Oulu, Oulu 90014, Finland
6. Department of Software Engineering, Lappeenranta University of Technology, Lappeenranta 53850, Finland
7. Department of Computer Science and Software Engineering, Penn State Behrend, Erie, PA 16563, USA

Received September 24, 2025; accepted April 6, 2026

E-mail: j.a.khan@herts.ac.uk

© Higher Education Press 2026

Abstract

Quantum Software Engineering (QSE) has emerged as a research direction practiced by tech joints. Quantum developers face challenges in optimizing quantum computing and QSE concepts. Developers use Stack Overflow to discuss quantum challenges using specialized tags to label posts. These tags often refer to technical quantum aspects. Categorizing quantum practitioners' questions by concept can help identify common QSE challenges. We conducted studies to classify quantum developers' questions into various challenges. We extracted 2,829 developers questions from Q&A platforms using quantum-related tags. The posts were analyzed to identify frequent quantum-related challenges and develop a novel grounded theory. The challenges identified include Tooling, Theoretical, Learning, Conceptual, Errors, and API Usage. Through content analysis and grounded theory, the developers' discussions were annotated with commonly reported quantum challenges to develop a ground truth dataset. ChatGPT was used to validate human annotations and resolve disagreements. Various fine-tuned transformer algorithms, including BERT, DistilBERT, and RoBERTa, were used to classify developer discussions into commonly reported quantum challenges. We achieved an average accuracy of 95% with BERT DistilBERT algorithms, compared to fine-tuned Deep and Machine Learning (D&ML) classifiers, including Feedforward Neural Networks (FNN), Convolutional Neural Networks (CNN), and Long Short-Term Memory networks (LSTM), which achieved accuracies of 89%, 86%, and 84%, respectively. The proposed Transformer-based approach outperforms the previous D&ML-based approach with a 6% increase in accuracy by processing actual developer discussions, i.e., without data augmentation. Furthermore, we applied SHAP (SHapley Additive exPlanations) to provide model interpretability, revealing how specific linguistic features drive predictions and enhancing transparency in the classification process. These improved research findings can help quantum vendors and developers' discussion forums to better organize developers' discussions for improved access and readability.

Keywords

quantum software engineering; repository mining; developer forums; stack overflow; transfer learning; natural language processing

1 Introduction

Quantum computing leverages quantum mechanics principles to describe the behavior of particles at the atomic and subatomic levels. Unlike classical computers, which use bits (0 s and 1 s), quantum computers use quantum bits or "qubits" to process information [1]. Superposition allows qubits to exist in a combination of 0 and 1 states simultaneously, increasing the computational power of quantum systems [2]. Google [3], IBM [4], and Rigetti [5] have demonstrated the ability to create qubits in controlled environments, enabling practical quantum computing. This provides opportunities to solve complex computational problems requiring parallel computation, which is challenging for classical computers. It can resolve complex issues in big data analytics, financial systems, national security, chemistry, cryptography, health analytics, and medicine [6].

Recently, there has been an increasing interest from the research community in proposing various approaches across different QSE activities [7]. Where QSE is a discipline emphasizing designing,

developing, and optimizing software for a wide range of applications [8]. QSE processes must be aligned with hardware advancements to leverage the capabilities of quantum computing effectively. Developing methodologies, processes, tools, and algorithms is crucial for quantum computing domains [9]. Quantum programming languages like Q# [10], Scaffold [11], and Qiskit [12] exist for developing quantum applications. Microsoft, Google, and IBM have developed cloud platforms that support the development of quantum software. IBM's platform enables the execution of applications on quantum systems. Developers create quantum software for domains such as chemistry and machine learning [13]. Given the importance of applications, software engineering processes are required for large projects. The QSE was coined to meet the demands for large applications [9]. Initial approaches for quantum requirements engineering [14], quantum software architecture [15], Quantum programming [16], Quantum Development [17], and Quantum Testing [18] are emerging. Software practitioners use developers' discussion forums to extract information and address challenges associated with software evolution [19, 20]. For example, researchers have analyzed Q&A platform discussions to identify challenges related to software development methodologies [21]. Also, De Dieu et al. [20] explored Q&A for extracting software architecture related information. Similarly, Li et al. [22] explored Q&A for extracting requirements-related information. Moreover, El-Aoun used Q&A to identify frequently occurring challenges to the quantum developers community [13]. These forums offer resources, communities, and opportunities for problem-solving. Q&A are valuable for sharing experiences in less-resourced areas, such as QSE, blockchain, etc. They enable collaboration, updates on trends, and collective problem-solving. The developers' posts in these Q&A platforms are categorized based on specific tags, which we believe are dificult for developers to remember. Particularly, the more emerging and comparatively new domains, i.e., QSE. Therefore, this motivated us to contextually analyze the developers' questions to identify their best possible category and add it to the existing tags associated with it from improved searching and visibility in the Q&A forums, which is aligned with previous research [23].

In this study, we evaluate developer discussions on QSE across various Q&A platforms like Stack Overflow (SO), Quantum Computing Stack Exchange (QCSE), Computer Science Stack Exchange (CSSE), and Artificial Intelligence Stack Exchange (AISE) to understand their contextual category, which are often reported as challenges by software quantum practitioners. This aids quantum developers in finding possible solutions to specific QSE-related challenges on these platforms that are reported by other quantum practitioners and are frequently organized by tags such as qiskit, qcl, qutip, qubit, and TensorFlow Quantum. For this purpose, we extracted 2829 developers' questions using quantum-related tags, following El-Aoun et al.'s method [13]. These developers' posts were manually critically analyzed to identify frequently discussed quantum challenges and develop a novel grounded theory, which was cross-validated with that of El-Aoun et al. [13] who identified QSE challenges using topic modeling. The frequent challenges include Tooling, Theoretical, Learning, Conceptual, Errors, and API Usage. Next, using content analysis and the novel QSE grounded theory document, the developers' discussions in the dataset were annotated with common quantum challenges to create a ground-truth dataset for the D&ML and transfer learning (TL) algorithms. Moreover, to validate the annotation process, ChatGPT was utilized to annotate developers' discussions. Additionally, another cycle was run with ChatGPT to resolve disagreements between the human annotators and ChatGPT, resulting in a conflict-free dataset. Finally, in this approach, in addition to the various fine-tuned D&ML classifiers, fine-tuned transformer learning algorithms, including BERT, DistilBERT, and RoBERTa, were used to contextually analyze developer discussions and classify them into common quantum challenges.

We extended the work published in the Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering [24], with the following additions.

- In addition to the D&ML experiments, we aim to propose a more generalised and robust approach by employing fine-tuned transformer-learning algorithms, achieving comparatively better performance in classifying developers' feedback into fine-grained quantum types by utilising transformers' contextual embeddings and pretrained knowledge.
- We extended the introduction by adding more recent and relevant QSE references and motivation for the proposed approach.
- We critically analysed developers' feedback using a content analysis approach to identify the frequency of quantum-related concepts in the Q&A forums.
- We extended the QSE dataset annotation using ChatGPT by adding various prompts to improve the understanding of complex annotation processes.
- We fine-tuned state-of-the-art transformer algorithms for improved accuracy and compared their performances with fine-tuned D&ML algorithms. Additionally, various confusion matrices are developed to demonstrate the performance of fine-tuned transformers and D&ML algorithms in achieving lower rates of false positives and negatives.
- We used SHAP as an explainability layer for TL models in the QSE domain, providing global and local interpretability to reveal how specific linguistic features influence model predictions, thereby enhancing transparency and trust in automated classification systems.
- We extended and refined the discussion of the manuscript with more insightful implications of the proposed approach for quantum developers and software vendors.

This study extends both prior study [24] and the empirical investigation by El-Aoun et al. [13] in a focused and complementary way. El-Aoun et al. [13] primarily identified QSE challenges by analyzing Stack Exchange posts and GitHub issues using topic modeling, providing an empirical view of what developers discuss

but without an automated fine-grained classifier for assigning new discussions to challenge types. In prior work [24], the study constructed a labeled dataset of 2,829 Q&A discussions and trained fine-tuned D&ML models for challenge classification, where performance improvements relied substantially on textual data augmentation and the use of ChatGPT to validate human annotations and resolve disagreements. Building on these foundations, the present manuscript makes three key advancements: (i) we fine-tune TL-based models (BERT, DistilBERT, and RoBERTa) to achieve substantially improved classification performance without requiring synthetic augmentation, providing a more practical solution for emerging, data-scarce QSE domains; (ii) we formalize and report the ChatGPT-assisted negotiation process for resolving boundary-case annotation conflicts and producing a conflict free ground truth; and (iii) we conduct a comprehensive SHAP-based explainability analysis to reveal the most influential linguistic cues driving predictions, improving transparency and supporting actionable insights for quantum tool developers.

The paper is organized as follows: Section 2 reviews a related work. Section 3 outlines the research methodology. Section 4 details the classification and analysis of challenges faced in QSE. Section 5 explores automated methods for classifying developer discussions. Section 6 discusses the insights derived from developer exchanges on platforms like Stack Exchange. Section 7 concludes the paper and discusses possible future directions.

2 Related work

This section reviews the pertinent literature on quantum computing, QSE, and mining software repositories.

2.1 Quantum software development approaches

This subsection reviews the literature on software quantum computing approaches and techniques for quantum software development. Gill et al. [25] provided a systematic review of Quantum Computing (QC), which offers computational advantages over classical computing by leveraging quantum mechanical principles. They highlighted QC's potential to address complex problems in drug design, data science, clean energy, finance, industrial chemical development, secure communications, and quantum chemistry. Akbar et al. [17] proposed a perspective on QSE, outlining its lifecycle stages and providing a framework for quantum necessity engineering, software implementation, design, testing, and maintenance. El-Aoun et al. [13] explored challenging aspects of QSE, examining quantum code theory and the gap between quantum and classical computing. Vietz et al. [26] analyzed tools supporting quantum application development, providing categorization and analysis to aid developers in selecting suitable tools. Haghparast et al. [27] explored QSE challenges, mapped research challenges to the quantum computing workflow model, and identified directions for software engineering research.

2.2 Mining Q&A repositories

We elaborate on the approaches that mine Q&A repositories to enhance developers' understanding. Beyer et al. [23] proposed a method to classify developer questions in SO forums into seven categories. They gathered data from SO forums on Android developers' questions and classified them using ML classifiers and expressions. Treude et al. [28] studied how programmers ask and answer questions on SO, analyzing discussions to categorize question types and assess answers. Their findings show the utility of Q&A sites for code reviews and conceptual questions. Iftikhar et al. [29] proposed a deep learning approach for predicting correct answers in SO discussions. They extracted metadata and question/answer combinations, applied NLP techniques, and used keyword ranking for feature vectors. Their ensemble deep learning model surpassed state-of-the-art methods, improving the accuracy, precision, recall, and f-measure by 1.72%, 24.96%, 6.57%, and 16.62%, respectively. Zhu et al. [30] studied SO discussions, noting active participation in Q&A. They found a strong link between question comments and the response time. Similarly, [31] analyzed Q&A forum discussions to gain insights into software development methods. El-Aoun et al. [13] analyzed Q&A platforms to uncover challenges faced by quantum developers. Khan et al. [32] proposed an automated approach for extracting requirement-related information from forums, using ML and NLP to categorize discussions. Beyer et al. manually categorized 450 SO posts on Android app development [33]. They found "How to?" and "What is the problem?" were common, with issues related to the User Interface and core elements. Their study revealed correlations between the problem and question types.

2.3 Recent advances in TL-based text classifications

TL has significantly advanced NLP-based text classification by enabling models pretrained on large corpora to be effectively adapted to domain-specific tasks. Recent evidence shows that transformer-based TL models often outperform traditional ML and DL approaches across a broad range of classification settings [34]. In RE, TL has been successfully applied to automate the identification and structuring of requirements-related information. For example, Araujo et al. [35] employed BERT to identify requirements-related information and reported improvements over prior approaches. Sainiani et al. [36] fine-tuned BERT to extract requirements from business contracts and classify them into requirement types, achieving stronger performance than conventional ML/DL baselines. Deshpande et al. [37] further demonstrated that fine-tuned BERT models can support RE analysis by identifying dependency relations within textual requirements. In addition, Hey et al. [38] proposed NoRBERT by fine-tuning BERT to classify functional and non-functional requirements using PROMISE datasets, highlighting the practical value of contextual language models for software engineering text.

TL has also shown strong effectiveness in mining software feedback and issue-related text. Henao et al. [39] used BERT-based TL to detect bug reports and feature requests from tweets and app reviews, reporting improved precision/recall over baseline methods. Kaur et al. [40] proposed Bert-RCNN, combining BERT with an RCNN architecture to improve bug-report classification by handling polysemy in app reviews. Bilal et al. [41] similarly demonstrated that fine-tuned BERT models outperform bag-of-words approaches for

review classification, reinforcing the benefit of TL in noisy, user-generated text. Finally, Devlin et al. [42] established BERT as a benchmark TL model by introducing bidirectional transformer pretraining that enables robust fine-tuning for downstream classification tasks. Motivated by these advances, our work adopts transformer-based TL to better capture the contextual semantics in QSE developer discussions and to improve classification accuracy over conventional D&ML baselines.

2.4 Comparison with existing literature

The study complements mining software repositories and analyzing developer discussions to improve practitioner understanding. Inspired by El-Aoun et al. [13] and Beyer et al. [23], our approach differs from previous studies. El-Aoun et al. [13] used topic modelling on Q&A forums to identify challenges, while Beyer et al. [23] classified developers' comments related to Android development into fine-grained categories. We critically analyzed QSE discussions from multiple forums to develop a taxonomy, validating it against El-Aoun et al. [13]'s findings. In our previously published paper on QSE [24], we fine-tuned ML and DL classifiers to classify QSE discussions into frequently occurring challenges. However, due to the comparatively emerging domain of QSE, the performance of D&ML algorithms was relatively low due to limited data availability in the discussion forum. Still, to counter this, we employed a data augmentation approach to improve the D&ML performances at the cost of adding additional complexity and risk of training the algorithms on synthetically generated data. In the proposed approach, we leverage the power of TL algorithms by fine-tuning them for the QSE problem to better understand the context of developers' discussions and classify them into more closely related QSE challenges. Moreover, we employed existing explainability approaches with the fine-tuned transformer to enhance the interpretability of the transformer algorithms to understand the features frequently used by TL algorithms when deciding on a particular QSE challenge type. Additionally, unlike the Beyer et al. [23] approach, we aimed for a generalizable approach using multiple QSE forums. We employed ChatGPT to validate coder disagreements and automate annotation. Table 1 compares existing approaches.

3 Proposed research methodology

In this section, we first elaborate on the research questions to answer the proposed research methodology. Next, we describe the proposed research approach, which includes the research dataset, novel grounded theory, content analysis, and various classifiers used to classify developer discussions into frequently reported challenge types.

3.1 Research questions

This study aims to develop an end-to-end automated methodology for systematically classifying developer discussions into identified challenges using fine-tuning D&ML and transfer algorithms. The proposed approach will empower QSE developers with structured resources, enabling them to find relevant information about the quantum challenges they face. To operationalize and validate this goal across the full pipeline (challenge identification, dataset construction, model evaluation, and interpretability), we formulate the following research questions:

RQ1: What do quantum developers discuss about various challenges in different developers' discussion forums?

RQ2: What frequent QSE challenges can be identified from the developers' discussion?

RQ3: Can ChatGPT work as an annotator and negotiator in developing a ground truth for D&ML and transformer classifiers?

RQ4: Do fine-tuned transformer algorithms outperform fine-tuned D&ML algorithms in classifying actual developers' discussions into various QSE challenges?

RQ5: Does the SHAP explainable approach enhance the interpretability of fine-tuned transformer models in understanding classification decision-making?

In pursuit of RQ-1, we aim to analyze developer questions in various forums, including SO, QCSE, CSSE, and AISE, to identify common patterns in how quantum developers express concerns or challenges regarding QSE. This will result in a novel grounded theory to identify frequently occurring challenges related to QSE. RQ2 utilizes the grounded theory and content analysis approach to develop a ground truth for RQ4 by manually annotating developers'

Table 1 Summary of methodologies for automated developer question classification

Ref.	Algorithms used	Proposed methodology	Dataset used and size	Key contributions	QC/QSE?
Beyer et al. [23]	RF, SVM	Automated classification via taxonomy harmonization (7 categories), regex/ML.	1,000 Android SO posts	Improved question search/browsing for developers.	No
Vietz et al. [26]	–	Quantum dev tools taxonomy + comparison framework.	–	Structured quantum tech selection for developers.	Yes
Khan et al. [21]	LDA	Mixed-methods analysis (13,903 posts) with topic modeling.	13,903 SO/SESE/PMSE posts	Identified common dev approaches/challenges.	No
El Aoun et al. [13]	Topic Modeling	QSE challenge identification via Stack Exchange/GitHub.	Stack Exchange + GitHub	Highlighted QSE-specific challenges.	Yes
Proposed work	**FNN, CNN, LSTM, GRU, RNN, BERT, RoBERTa, DistilBERT**	**Classification of 2,829 quantum dev questions + ChatGPT validation + data augmentation.**	2,829 Q&A posts	QSE challenge taxonomy + LLM validation + enhanced classification.	**Yes**

discussions into various frequently occurring challenges. For RQ3, we seek to determine whether ChatGPT can annotate developers' discussions about QSE compared to human annotators and explore its use as a negotiator to resolve conflicts between annotators. This aims to overcome the manual complexity of annotating developer datasets for classification tasks. For RQ4, we aim to evaluate and compare various D&ML and TL algorithms by exploring different feature engineering and hyperparameter tuning approaches to fine-tune these algorithms and identify their efficacy in classifying developers' discussions into QSE challenges identified through RQ1. Moreover, in the previously published work [24], the D&ML algorithms revealed limitations in capturing the complex linguistic nuances of QSE-related developers' discussions by classifying them into frequently occuring QSE challenges due to limited data. In the proposed approach, we fine-tuned TL models, specifically BERT, DistilBERT, and RoBERTa, and evaluated their performance in classifying developers' discussions from Q&A forums for QSE challenges, compared to D&ML algorithms without data augmentation. Finally, RQ5 investigates the application of SHAP to these TL models to provide transparent and interpretable results.

3.2 Research method

The proposed research methodology for classifying developers' discussions from various social media platforms into QSE challenges comprises four main phases, shown in Fig. 1; each methodological step is elaborated below.

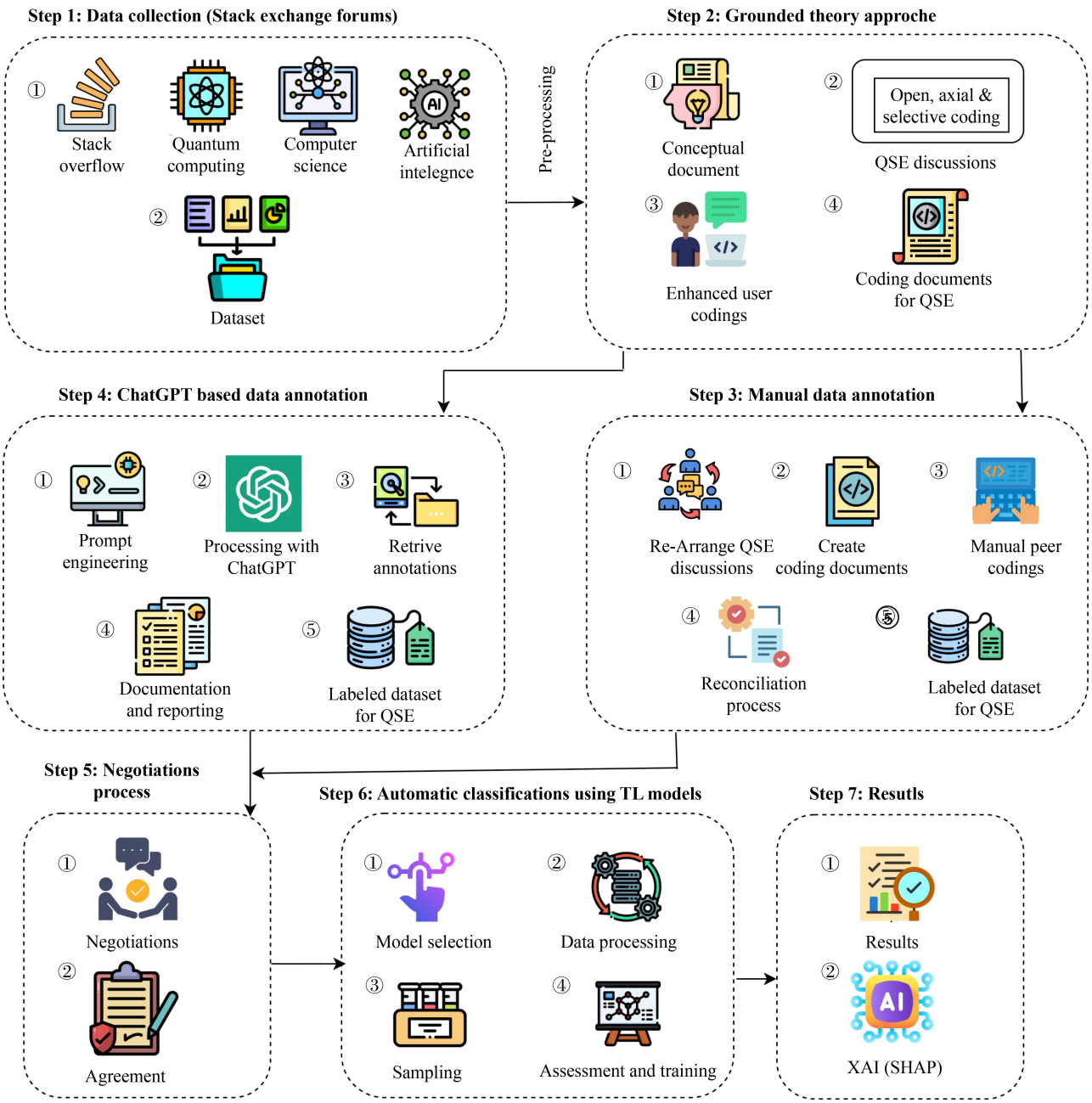


Fig. 1 Proposed workflow for mining and classifying QSE-related discussions using transformers and SHAP

3.2.1 Research data gathering and curation

For the proposed approach, a research dataset on QSE is developed by systematically collecting developers' discussions from SO, QCSE, CSSE, and AISE forums due to their importance in software engineering literature [19,23]. Recently, El Aoun et al. have explored these platforms for identifying quantum-related challenges using topic modeling [13]. Also, [21] explored these Q&A forums for identifying challenges encountered by software developers when implementing various software methodologies. These Q&A platforms have proved to be an important source for improving various software engineering activities [19]. As of June 2024, SO had over 23 million, AISE had 73,124, QCSE had 23,740, and CSSE had 140,047 registered users. The SO repository, with over 24 million questions and 35 million answers, is the largest question-and-answer platform for software developers. We focused on collecting developers' discussions about quantum computing, programming languages, algorithms, and tools from these platforms, aiming to help quantum developers better identify challenges by adding a challenge type to the existing quantum tags.

We navigated the SO, AISE, QCSE, and CSSE forums using tailored queries to extract developer discussions relevant to QSE. For SO, queries targeted questions tagged with identifiers like post-quantum-cryptography, q#, quantum-computing, qiskit, qcl, qutip, qubit, and TensorFlow quantum, based on El-Aoun et al. approach [13]. These queries retrieved accepted and non-accepted answers, ensuring a comprehensive view of QSE discussions. Using SEDE, data from August 2008 to August 2024 were extracted. The SO had 627 questions with 282 accepted answers. For QCSE, CSSE, and AISE, queries focused on tags like programming, classical computing, and Q#. QCSE had 1,806 questions with 500 accepted answers, CSSE had 379 questions with 124 accepted answers, and AISE had 17 questions with 4 accepted answers, as shown in Table 2. We note that AISE contributes a relatively small number of posts (17 questions), which limits platform-specific representativeness. This low volume likely reflects the emerging nature of QSE discussions on AI-focused forums and the tendency for quantum programming questions to concentrate in specialized communities (e.g., QCSE) or other channels.

3.2.2 Grounded theory approach

In the previous step, developers' QSE discussions were collected from various Q&A forums as raw data. To make this data usable, we manually selected a sample of 300 developers' discussions to critically analyze them and identify frequently occurring challenges related to QSE, and developed a grounded theory. For this purpose, we employed Corbin's grounded theory framework [43] to develop a coding guideline, which works as an input for preparing an annotated dataset for D&ML algorithms. Grounded theory constructs a theoretical framework based on recurring evidence from the QSE dataset. The novel ground theory for QSE involves frequently mentioned challenges that quantum practitioners face in Q&A forums. The frequently reported challenges identified are Tooling, Theoretical, Learning, Conceptual, Errors, and API Usage. The identified quantum concepts were verified with challenges identified by El-Aoun et al. [13] using a topic modeling approach. Compared to El-Aoun et al. [13], we discarded three challenges: discrepancy, review, and API change, due to insufficient evidence in developers' discussions. In our initial grounded-theory sample, these three categories occurred only rarely (each appearing in less than 4% of the sampled discussions), and therefore were not retained as standalone challenge types in the final taxonomy. Some concepts recur frequently; they are merged into recurring challenges. For example, discussions about unexpected results overlap with error troubleshooting and are represented with the error quantum concept. Discussions related to "API change" are grouped in the conceptual category. The "Review" challenge type can be intertwined with the QSE "Conceptual" or "API usage" categories. The quantum grounded theory process was developed iteratively with discussions by the first, second, and third authors of the paper. The document helps annotators label the raw developer discussion for D&ML classifiers and minimize discrepancies among coders when annotating the dataset for D&ML learning experiments.

3.2.3 Analyzing content manually (Reviews annotation with humans and ChatGPT)

After developing grounded theory, raw developer discussions from Q&A platforms must be annotated to make a useable dataset for D&ML and TL classifiers. Using the content analysis approach [44] and a QSE grounded theory document, developers' discussions are manually annotated. For this purpose, 2,829 developers' discussions on QSE collected from Q&A platforms were added in a Microsoft Excel document to be annotated by the first and third authors of the paper using grounded theory document and content analysis approach. Moreover, content analysis is challenging and time-consuming due to human annotator involvement [45]. Therefore, to

Table 2 Dataset for TL experiments

Topic	Total questions	Tag set	Data source
Stack overflow	627	post-quantum cryptography, q#, quantum-computing, qiskit, qcl, qutip, qubit, TensorFlow-quantum	Stack Exchange
Quantum computing	1806	programming, classical computing, q#, qiskit, cirq, ibm-q-experience, machine-learning, qutip	Stack Exchange
Computer science	379	quantum-computing	Stack Exchange
Artificial intelligence	17	quantum-computing	Stack Exchange
Total	**2829**	–	–

possibly automate and improve the manual annotation process, the same comments were annotated using ChatGPT to compare its performance with human annotators, as an alternative data annotation step [46]. In this study, we used ChatGPT-3.5 to support annotation and to help resolve disagreements during the ground-truth construction process. Annotation forms from human coders were merged to identify interceding agreements and cohesion kappa to assess ChatGPT's applicability in annotating QSE-related comments. To resolve conflicts, another round was conducted between human coders and ChatGPT to agree on comment annotations. The detailed process is described in Section 4.2.

3.2.4 Classifying developer discussions using D&ML and TL classifiers

After annotating QSE-related developer discussions, we assessed existing fine-tuned D&ML and TL classifiers in categorizing these discussions into QSE-related challenges identified via a grounded theory approach. These QSE challenge categories can be used as an additional tag to the existing QSE-related tags, improving QSE-related knowledge identification for software developers on the existing Q&A platforms. For this purpose, we performed the following: A preprocessing pipeline removed special characters, URLs, and symbols using regular expressions. NLP techniques, like stemming and removing stop words, enhanced readability for D&ML algorithms. Feature engineering approaches, such as CountVectorizer and TFIDF, were employed to evaluate their effectiveness in classifying discussions. For deep learning classifiers, various feature values were experimented with and fine-tuned for improved results. Data-balancing methods, such as under-sampling and over-sampling, address data imbalance across various challenges.

For the final reported models, we applied random over-sampling of minority classes to mitigate class imbalance and improve model stability, while preserving the original text instances (i.e., without generating synthetic posts). Data augmentation enhanced classifier performance, including random word deletion with a 20% retention probability, random word swapping up to two times, and random insertion of synonyms. For the proposed approach, we removed the experimental steps for ML and DL experiments due to the length of the paper. However, the details are listed in the previously published paper [24]. Moreover, for the transfer-learning experiments, we first selected the existing transfer-learning algorithms based on their performance on textual data analysis. Next, we perform preprocessing and fine-tuning to achieve improved results using TL algorithms. Additionally, the instances across different categories are highly imbalanced. Therefore, to balance it, we applied a data sampling approach. A cross-validation approach is used to train and validate D&ML and transfer classifiers for generalized results. D&ML and transformer models' performance was evaluated using the metrics of precision, recall, F1-score, ROC, and learning curves. Finally, we employed the SHAP explainable approach to explore the block-box nature of transformer algorithms and identify frequently used features by them for decision-making.

4 Processing developers discussion

This section outlines the process of identifying the various challenges that quantum developers face while developing quantum-related applications. In addition, we shed light on how to annotate developer discussions collected from various Q&A platforms into the challenges identified previously using the grounded theory approach. Below, we elaborated on the process.

4.1 Statements of developers

Organizing existing developers information on these Q&A platforms can help developers identify relevant information promptly and more accurately. QSE is a new area where quantum developers frequently face challenges [13]. Therefore, we analyze developers' discussions to identify the frequent challenges that quantum developers face, including tooling, theory, learning, concepts, errors, and API usage. These categories can be added to the existing searching algorithms for improved identification of challenge-based developer posts. The related quantum codes identified during the annotation emerged as concepts, resulting in a novel quantum theory for software developers. Concepts in the quantum coding guideline were identified by their consistent presence in discussions and relevance to the research approach. If a quantum concept appears less frequently in discussions, it merges with a related quantum concept. For example, the "API change" concept can integrate into the "Conceptual category," aligning with the definition where developers ask about the API's working. Below, we discuss the identified QSE-related concepts.

4.1.1 Conceptual

The code "Conceptual" is assigned to developers' discussions on Q&A platforms focusing on understanding the background, limitations, and concepts of quantum programming APIs. Analysis shows that quantum developers discuss quantum computing history or debate the feasibility of certain concepts. Researchers categorize discussions into conceptual categories if questions contain terms like why, is possible [33], what [47], how/why [48], and understanding concepts [23]. The definition for categorizing discussions aligns with approaches by Beyer and Pinzger [33], Rosen and Shihab [47], Allamanis and Sutton [48], and Beyer et al. [23]. An example is in Row 2 of Table 3.

4.1.2 Theoretical

The code "Theoretical" is assigned to Q&A platform discussions on quantum programs, algorithms, and principles. Developers may inquire about the mathematical foundations of quantum mechanics or seek explanations for quantum algorithms. For instance, a developer might ask how superposition or entanglement affects quantum algorithm design, as shown in Table 3, row 3. It discusses the theoretical implications of practical quantum computing on public-key cryptographic algorithms and complexity theory. El-Aoun et al. [13] identified that developers often discuss theoretical problems related to quantum computing in Q&A discussions using a topic modelling approach.

Table 3 Quantum concepts with examples from Q&A forums

Concept	Developers' discussion examples
Conceptual	Quantum Computing and Encryption Breaking: "I read that Quantum Computers can break most encryption... How? I get lost at quantum bits being 1, 0, or both. Can someone explain in plain English without the math?"
Theoretical	NP-hard Cryptography: "Are there public key algorithms based on NP-complete problems rather than factorization/discrete logs, in case quantum computing becomes practical?"
Tooling	QASM Simulation Issues: "Using IBM Quantum Experience: Dragging gates is slow, no Toffoli gate. Need QASM editor/documentation. Found these links but unclear which QASM version IBM uses..."
Learning	Book Request: "Looking for a book covering Shor's algorithm, McEliece cryptosystem, Lattice-based crypto, and Discrete logarithms. Any recommendations?"
Error	Q# Build Problem: "Getting error QS1001: Microsoft.Quantum.Canon.dll not found. Missing.nuspec file. Nuget restore didn't help. Solutions?"
API Usage	Break Equivalent in Q#: "How to exit a loop when condition met? In C#: if(i==3) break. What's the Q# equivalent?"

4.1.3 Tooling

This category pertains to developer discussions on tools and software usage in quantum development. The code "tooling" is assigned to developers' discussions on Q&A platforms that discuss possible quantum tools, seek expertise, or help with quantum-related tools. Quantum computing is an emerging discipline where vendors develop new tools to solve quantum-related problems. During annotation, quantum developers often seek guidance on using specific software tools or frameworks for quantum development. An example is shown in Table 3, Row 4. This category aligns with the El-Aoun et al. [13] approach, which identified "Tooling" as a frequently occurring quantum challenge developers discuss in Q&A forums.

4.1.4 Learning

In the developer discussions dataset where quantum practitioners seek resources about quantum computing from the Q&A community, we assigned the code "learning." This category includes requests for learning resources, tutorials, and references for quantum computing. Developers often seek recommendations for books, online courses, or other resources to enhance their understanding of quantum programming languages and algorithms. For instance, a quantum developer inquires about the best resources for learning Q# or mastering Grover's search algorithm, as shown in Row 5 of Table 3. The "Learning" category for the quantum classification taxonomy aligns with Beyer et al. [23] and El-Aoun et al. [13] "Learning" category identified for android-related topics and quantum challenges using topic modelling, respectively. Also, Allamanis and Sutton [48] identified learning a language/Technology category with different purposes and intentions.

4.1.5 Errors

Quantum computing is a new research and development area with limited expertise in quantum programming compared to traditional paradigms. We identified that quantum developers often seek help resolving errors while developing quantum applications. We assigned the code "errors" to developers' posts on Q&A platforms where quantum practitioners ask for solutions to errors. During manual annotation, we found that quantum developers often encounter challenges related to quantum gates, qubit decoherence, and compatibility issues with quantum hardware. For example, a practitioner asked for help debugging quantum circuits or understanding error messages from a quantum simulator, as shown in row 6 of Table 3. This category aligns with Beyer et al. [23], El-Aoun et al. [13] "Errors", and Beyer and Pinzger [33] "Error and Exception Handling" category.

4.1.6 API usage

As quantum computing evolves, numerous APIs have been developed for various functionalities that quantum developers can utilize. The code "API usage" is assigned to developers' discussions in the Q&A seeking help in efficiently implementing a particular Quantum-related API. During manual annotation, it was noted that discussions focused on using APIs for quantum programming tasks. Developers seek guidance on integrating quantum computing platforms and services into their projects using APIs. For example, a developer may inquire about best practices for interfacing with the IBM Quantum Experience API or accessing the Rigetti Forest API for quantum program execution, as shown in Table 3, row 7. This category aligns with those of El-Aoun et al. [13] and Beyer et al. [23] API usage category, Allamanis and Sutton's [48] "Do not work" category, and Beyer and Pinzger's [33] "How to" category.

4.2 Developers statement labeling

To annotate developer discussions from Q&A platforms, we used content analysis [44] and QSE grounded-theory document, examining each developer's question to identify their quantum challenge category. The goal is to develop a truth set for training and testing the fine-tuned D&ML and transfer algorithms to categorize developer questions into quantum categories automatically. The annotation process is described below:

4.2.1 Annotation of developer discussions

Annotation was conducted in two steps. First, the first and third authors individually annotated 2,829 developers' discussions from Q&A forums. Second, ChatGPT annotated the same dataset to compare its performance with human coders. Human annotators were given a coding guideline and form with developer comments. Coders assessed the title and content to determine the quantum category: tooling, theoretical, learning, conceptual, errors, or API Usage. Figure 2 shows an overview of the coding process. Developers' questions were grouped by platform, including answers for clarity.

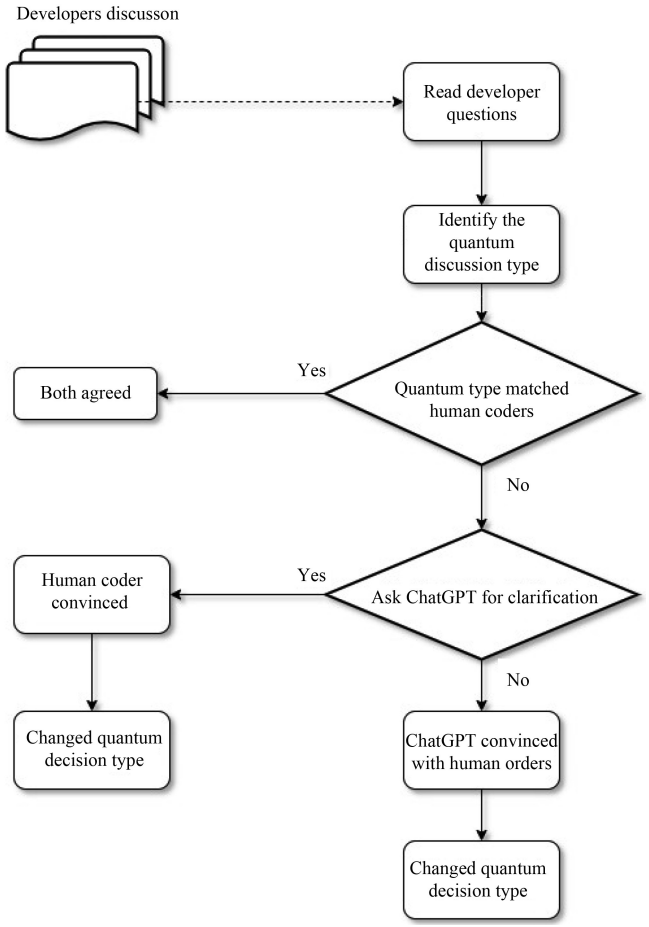


Fig. 2 Annotation and negotiation process with ChatGPT

The annotation process was iterative, as shown in Fig. 1. The average annotation time was 44 working hours. The coders start and resume annotation at any time. After individual annotation, the results were combined to analyze intercoder disagreements. The intercoder agreement was 90.81%, with Cohen's kappa at 0.46 (moderate agreement), with 95% confidence intervals [0.43, 0.49]. Conflicts were resolved through discussion, with a senior software researcher (second author) intervening for persistent disagreements. To validate the annotation process, we used ChatGPT to annotate developers' discussions into quantum categories and compared its performance with human annotators. We trained ChatGPT with definitions of quantum challenge types from novel quantum-grounded theory using the existing prompt. We supplied each developers' discussion to ChatGPT and identified its possible quantum challenge type with a rationale. We recorded the number of discussions where manual annotators and ChatGPT agreed and disagreed. Table 4 shows a summary of annotated developer discussions, and Table 5 shows detailed annotations by human coders and ChatGPT, agreeing on 2,547 discussions for the same quantum challenge category and disagreeing on 282, resulting in potential conflicts. The Cohen's kappa was 0.46, indicating moderate agreement between human coders and ChatGPT on the Cohen's kappa scale.

A negotiation process resolved conflicts between human coders and ChatGPT. For conflicting developer discussions on the Q&A forum, a second round with ChatGPT was initiated. A generic query asked for elaboration on selected quantum categories. This discussion continued until a consensus was reached. Among 282 conflicting discussions, ChatGPT agreed with human annotators on 173 questions after reasoning. Conversely, human coders agreed with ChatGPT on 109 developer feedback after receiving justification. This process curated a conflict-free dataset for QSE, as shown in Table 5. This step enhances the labelled dataset's quality by validating human annotation with ChatGPT, improving the effectiveness of subsequent D&ML and TL classifiers in identifying quantum developer challenge types.

■ 4.3 Frequency of the developer's discussions on Q&A platforms

Before subjecting the developer discussions to fine-tuned transformer algorithms, it is essential to identify the frequency of developer instances across each quantum challenge category to achieve better results. A detailed analysis aids in understanding the data, quantum trends, challenges, and recent developments. We utilized content analysis to examine the frequencies of developer discussions across categories, providing insights for quantum software vendors and developers. The goal is to introduce various types of quantum discussions, including tooling, theoretical, learning, conceptual, error analysis, and API usage, to assist quantum vendors in understanding trends and difficulties faced by practitioners. The most frequent type of developer question on Q&A platforms is the "Error" quantum

Table 4 Summary of annotated developer discussions on quantum computing

Q_Title	Q_description	Q_Type
Quantum computing & encryption break	I read that Quantum Computers can break most encryption in minutes. How? I get lost at quantum bits being 1, 0, or both. Can someone explain in plain English without the math?	Conceptual
Qubit density to bloch vector	Given a 2x2 density matrix, how to compute the Bloch sphere point? State —0〉 - —1〉 has matrix [[0.5,-0.5],[-0.5,0.5]] and should be on X axis. Matrix [[0.5,0],[0,0.5]] should be at origin.	Theoretical
What is quantum computing?	In physics, particles in multiple states simultaneously. In computing, bits as 1, 0, both, or NULL? Applications to processors, programming, security? Any practical implementations?	Learning
floor/ceil in QCL	What do QCL's floor() and ceil() operators do exactly? They seem related to math operations but need clarification.	Tooling
QuTiP integration failure	Getting errors solving Lindblad equation. Increasing nsteps doesn't help. Changing time values (np.linspace) has no effect. Need solution.	Errors
Can forest crack crypto?	Can Rigetti Forest break public-key crypto (e.g., Bitcoin) in reasonable time? If yes, show solution using pyQuil.	API Usage

Table 5 Details of human and ChatGPT annotations

Quantum category	Human freq	ChatGPT freq
Tooling	600	611
Theoretical	414	446
Learning	159	158
Conceptual	606	597
Errors	803	809
API Usage	247	208

Total number of questions: 2,829 (Human), 2,829 (ChatGPT)

Disagreement analysis: There are 282 disagreements between human coders and ChatGPT. The number of learning experiences aligns with this count. In each category, there are disagreements where the human coders, for example, incorporate the learning category where ChatGPT utilizes tools. Overall, disagreements exist across various categories.

type, comprising 28.80% of discussions (815 questions), as shown in Table 6. Quantum software development is emerging, with limited resources and tools. Developers often encounter challenges and seek assistance from open-source communities, primarily related to troubleshooting, debugging, and addressing technical issues in quantum software. The second most frequently reported quantum challenge is conceptual, accounting for 21.56% of discussions (610 instances). Quantum researchers, investors, and vendors are increasingly interested in proposing new research methodologies and techniques for quantum problems. Developers engage in conceptual discussions to understand the theoretical and fundamental principles behind software applications. Practitioners discuss various concepts related to quantum programming, debating the feasibility of certain quantum concepts. This presents opportunities for software vendors and investors to consider practitioners' discussions about the effectiveness of proposed quantum methodologies and overcome challenges for better productivity. The third most important category identified through content analysis is Tooling, constituting 21.06% (596) of the total discussions (2,829 developers' questions). Quantum developers frequently seek information on tools and software usage in quantum programming. As researchers introduce new tools to automate quantum functionalities, practitioners are likely to turn to open-source developers to share their experiences. This suggests that developers are actively exploring tools to enhance quantum software development. It presents an opportunity for quantum vendors to concentrate on tool usage and assist developers in better understanding the necessary functionalities. Such feedback could enhance the usability and developer experience with the available quantum tools. The next significant category is theoretical, accounting for 14.66% (415 developers' threads) of the total discussions (2,829), as shown in Table 6. Quantum developers engage in Q&A platforms to discuss complex theoretical concepts, algorithms, and principles for quantum-based applications. Quantum vendors can incorporate essential information and examples to improve understanding. The second least mentioned quantum challenge type is API Usage, making up 8.02% (227 developers' discussion questions). Quantum software development is intricate. Creating such applications from scratch is time-consuming and resource-intensive. Quantum developers often seek guidance on integrating APIs, reporting challenges in understanding documentation, and troubleshooting API-related issues. This helps quantum investors and vendors recognize the potential value in developing free APIs that extend quantum functionalities, saving developers' time and resources. Finally, learning emerged as the least-discussed quantum challenge type, comprising only 5.86% (166) of the total discussions (2,829 questions). Quantum developers seek recommendations for resources to enhance their understanding of quantum programming languages, algorithms, and related concepts. Identifying such information can help software quantum

Validation and Reconciliation of Annotations

Overview of Disagreements: Approximately 10% of developer discussions (282 out of 2,829) showed disagreements between human and ChatGPT annotations, mainly in boundary cases between *Tooling*, *Learning*, and *Conceptual* categories.

Resolution through Negotiation: In 38.7% of cases (109), human coders revised their labels to match ChatGPT's reasoning; in 61.3% (173), ChatGPT adjusted its label after reviewing human justification.

Example of Review Disagreement and Resolution:

```
- User Review: "It's been a while since I got into QCL but I ran into some
    difficulties by trying to figure out the functioning of measure regX or
    measure regX,m. What does this do on the quantum registers? And the m
    integer?"
- ChatGPT Annotation: Learning (intent to understand functionality).
- Human Annotation: Tooling (focus on QCL operator).
- Outcome: Humans agreed with ChatGPT after recognizing the query's educational
    intent. Final label: Learning.
```

Implication: This human-AI collaboration improves annotation quality by combining expert judgment with scalable pattern recognition, resulting in a more accurate and reliable dataset for training classifiers in quantum software engineering.

Prompts for QSE Challenges Detection Using ChatGPT

Example of QSE Categorization Prompt:

```
"Please categorize the following quantum developer discussion based on these
   challenge categories:
- Conceptual: Understanding the basic concepts of quantum programming and
   computing.
- Theoretical: Inquiries regarding the mathematical and theoretical foundations
   of quantum computing.
- Tooling: Issues related to the software tools, APIs, and quantum simulators
   used in quantum development.
- Learning: Requests for learning resources, tutorials, or guidance on quantum
   algorithms and programming languages.
- Errors: Problems encountered during the development process, such as errors
   in code or simulation issues.
- API Usage: Assistance with integrating or using quantum-related APIs in
   development.
```

Table 6 Finalized conflict-free annotated quantum dataset

Quantum category	Number of developers' discussions
Tooling	596
Theoretical	415
Learning	166
Conceptual	610
Errors	815
API Usage	227
Total	**2829**

vendors provide assistance on frequently occurring errors by embedding it into the existing quantum tools. Similarly, vendors can support the often-asked software quantum concepts in developer platforms. Moreover, crash courses can be offered on emerging technologies, such as software quantum computing, to boost software productivity.

5 Automated classifications of developer discussions using TL models

In the literature, mining developers' feedback on Q&A platforms significantly contributes to software evolution [19]. Researchers have mined Q&A platforms, particularly SO, for various purposes, such as developers' opinions about APIs [49], mining quantum programming [16], identifying challenges for software methodologies and popular topics [21], popular programming language developers [50], and bug severity prediction [51]. Manually identifying useful information for software developers and vendors is cumbersome and resource-intensive [19]. Complementary to these approaches, we mined developer discussions from Q&A platforms to identify frequently mentioned quantum challenges, aiming to provide opportunities for quantum vendors to focus on key areas developers struggle with or discuss most frequently. Also, the proposed approach can improve the searching mechanism of existing Q&A platforms by using the proposed classification approach to identify the category type and add to existing tags. To achieve a more robust and context-aware classification, in addition to the existing D&ML experiments, we experimented with the state-of-the-art TL algorithms to improve accuracy and low ratings of false positives and negatives. These advanced models are better suited for understanding the complex linguistic nuances present in the developer discussions. Moreover, we do not include the preprocessing, fine-tuning, data balancing, and detailed explanation of D&ML experimental results, as it is listed in the previously published paper. Therefore, in this paper, we focused more on the transformer-based experiments. Below, we elaborate on the experimental setup for applying these TL models and their subsequent steps.

5.1 Experimental setup

To ensure the reliability and reproducibility of the results, a robust computational environment was configured for all data processing and model training tasks. The primary development was conducted on a Macbook Pro M2, with experiments run within a Jupyter notebook environment. This setup allowed for iterative development and seamless integration with specialized Python libraries for NLP.

The software stack leveraged state-of-the-art libraries such as PyTorch was used as primary framework for building and trainings TL architectures, while the Hugging Face Transformers library provided the core implementation for the models and their tokenizers. For evaluation, Scikit-learn was employed to calculate performance metrics and implement cross-validation splits. All data manipulations and preparation was handled using Pandas and NumPy, and visualizations were generated with Matplotlib and Seaborn. This setup ensured optimal and consistent conditions for the computationally intensive task required by this study. A complete summary of system configurations is presented in Table 7.

5.2 TL models selection

Selecting the appropriate classification model is crucial for improved and generalized results. This study employs three state-of-the-art TL models based on the Transfer Architecture: BERT, DistilBERT, and RoBERTa. These models were chosen for their robust performance

Table 7 System configuration

Specification	Details
Hardware accelerator	NVIDIA CUDA-enabled GPU
Development environment	Jupyter notebook
Programming language	Python
Machine learning framework	PyTorch
NLP library	Hugging Face transformers
Data manipulation	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Evaluation tools	Scikit-learn

on a wide range of NLP benchmarks. Their pretrained nature allows them to leverage deep linguistic knowledge to effectively discern the context and semantics within the specialized domain of QSE forums.

5.2.1 BERT

As a foundational model, BERT [42] TL algorithm is selected to serve as a baseline, which proves to be a better choice compared to existing ML approaches [52]. Its key innovation lies in its bidirectional training, which allows it to process the entire context of a text simultaneously, rather than sequentially. This enables a deeper understanding of linguistic relationships within a sentence. This makes it a better choice for processing QSE-related developers' discussion posts, which contain lengthy textual information at times. For this study, the Bert base-uncased configuration was utilized, which consists of 12 encoder layers and a 768-dimensional hidden state. Its pre-training on large, general-purpose corpora provides a solid foundation for fine-tuning on the specialized vocabulary found in QSE discussions.

5.2.2 DistilBERT

DistilBERT was included to evaluate the performance of a more computationally efficient architecture [53]. This model is a condensed version of BERT, created using a technique known as knowledge distillation. It is approximately 40% smaller and significantly faster during inference while retaining the vast majority of BERT's performance. This makes it a compelling candidate for real-world engineering applications where resource constraints and response time are important considerations. Its architecture, while more compact, maintains the core principles of the Transformer, allowing for a meaningful comparison of the performance-efficiency trade-off.

5.2.3 RoBERTa

To evaluate a model featuring an improved pre-training methodology, RoBERTa was selected for this study. Developed by Facebook AI RoBERTa modifies the original BERT pre-training process by removing the next-sentence prediction objective and instead training exclusively with a masked language modeling (MLM) objective on a significantly larger text corpus [54]. Furthermore, it employs a dynamic masking strategy, where the masking patterns applied to the input tokens are varied during each training epoch. For tokenization, RoBERTa utilizes a Byte-Pair Encoding (BPE) vocabulary, which is particularly effective at handling the specialized technical terms found in QSE developer forums. These optimizations result in a model with a more nuanced understanding of language, making it exceptionally well-suited for fine-grained classification tasks of distinguishing between the subtle differences in developer discussions.

5.3 Data preparation and pre-processing for TL models

The preparation of the raw textual data from developer forums is crucial for effective analysis with TL models. This stage involved a systematic workflow to clean, format, and structure the data to align with the input requirement of the transfer architectures, ensuring that the semantic integrity of the developer discussions was preserved while optimizing them for the classification task.

5.3.1 Class label conversion

For the supervised classification task, the six categorical labels representing the QSE challenges (*Tooling, Conceptual, Errors, Theoretical, API Usage,* and *Learning*) were converted into numerical format. This is achieved by mapping each unique category string to a distinct integer index (0 through 5). This transformation is a standers requirement for DL and TL frameworks like PyTorch, as it allows the model's loss function to compute gradients effectively during training.

5.3.2 Model-specific tokenization

Unlike traditional NLP that requires aggressive cleaning, Transformer models utilize their own specialized tokenizers that are trained to handle near-raw text. The primary Pre-processing step, therefore, was a basic cleaning pass to remove artifacts like HTML tags or stray URLs. Subsequently, the cleaned text of each developer discussion was processed using the specific tokenizer corresponding to each model (i.e., *BertTokenizerFast*, *RobertaTokenizerFast*, and *DistilBertTokenizerFast*). This process involves three key actions: (1) converting the text into subword tokens based on the model vocabulary, (2) adding special tokens (e.g., *CLS* and *SEP*) required by the architecture, and (3) padding or truncating all sequences to a uniform length to enable batch processing. Attention masks were also generated to allow the model to differentiate between meaningful tokens and padding tokens.

5.3.3 Token length distribution analysis

To determine an optimal sequence length for padding and function, an initial analysis of the token length across all developers' discussions was conducted. The distribution revealed that the vast majority of the content could be effectively captured within a sequence length of 128 tokens. This value was chosen as the max_length parameter for tokenization, as it provides an excellent balance between retaining crucial textual information and maintaining computational efficiency during the model training and inference phases. The histogram in Fig. 3 highlights the right-skewed nature of the distribution, with most discussions falling below 200 tokens but a significant tail extending beyond that.

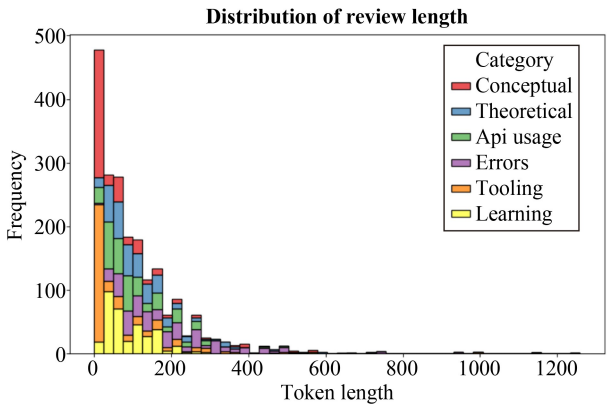


Fig. 3 Token length distribution across all categories

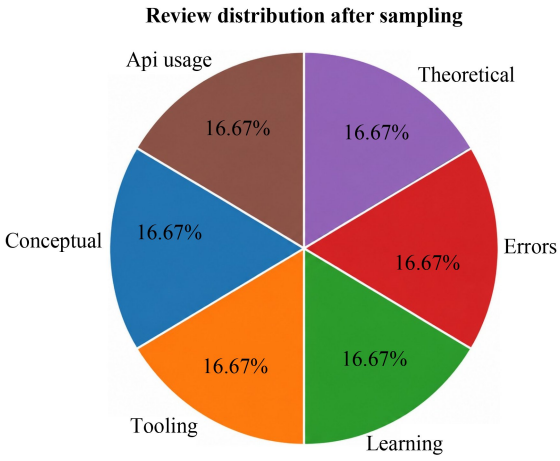


Fig. 4 Categories distribution after data balancing

5.4 Data sampling

The initial conflict-free dataset, while rigorously validated, exhibited a significant class imbalance, which is a common challenge in supervised ML [45,46]. An analysis of the class distribution, as shown in Table 6, revealed that some QSE challenge categories were far more prevalent than others. For instance, the 'Errors' category contained 815 developer discussions, whereas the 'Learning' category comprised only 166. Training a model on such an imbalanced dataset can introduce significant bias, causing the classifier to favor the majority classes while performing poorly on the underrepresented minority classes.

To mitigate this potential bias and ensure the model learns the features of each category with equal importance, a strategic up-sampling approach was employed. This technique balances the dataset by randomly duplicating instances from the minority classes until they match the sample count of the majority class. The *scikit-learn* library *resample* function was used to perform this task. The result of this procedure is a perfectly class-balanced training dataset, as illustrated in the distribution shown in Fig. 4. This balanced dataset forms the foundation for training the TL models, enabling them to develop a more robust and generalizable understanding of all six QSE challenge categories.

5.5 Training and fine-tuning of TL models

The pre-trained models were not used off-the-shelf: they were fine-tuned on the domain-specific QSE challenge dataset to adapt their general linguistic knowledge to the specific vocabulary and context of developer forums. To ensure the results are robust and to prevent overfitting, a 5-fold cross-validation strategy was employed. For each fold, the dataset was split into a training set (80%) and a validation set (20%), ensuring that every data point was used for both training and validation across the five iterations.

The fine-tuning process of each model was conducted over 30 epochs with a batch size of 16. We utilized the AdamW optimizer with a learning rate of 2e–5, which is a standard and effective choice for training Transformer models. A linear learning rate scheduler with a warmup phase was also implemented to help stabilize the training process in the initial epochs.

The training and validation progress for each model was carefully monitored. The learning curves for the BERT model, presented in Fig. 5, show a consistent decrease in loss and an increase in accuracy, indicating stable learning. In addition, all three models were successfully fine-tuned on the dataset without significant signs of overfitting.

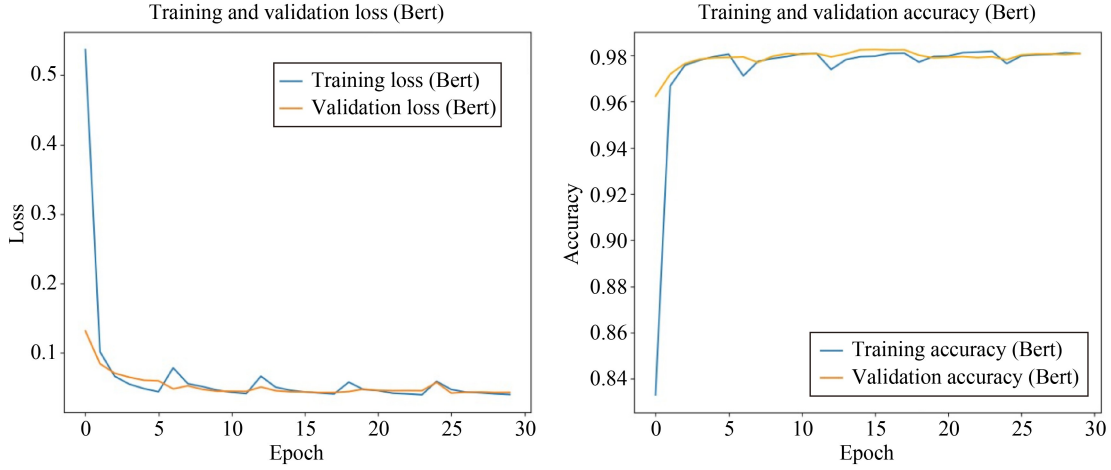


Fig. 5 Training and validation accuracy and loss for BERT model

5.6 Assessment and training

To train the TL models and evaluate their performance, a 5-fold cross-validation strategy was implemented. In this approach, the dataset is partitioned into five equal-sized folds. During each iteration, one fold is held out for validation and the remaining four folds are used for training. This process is repeated five times, with each fold serving as the validation set exactly once. This method ensures that every data point is used for both training and validation, providing a more reliable estimate of the model generalization performance that a single train-test split.

The effectiveness of the classifiers was assessed using a standards set of performance metrics:Precision, Recall, and F1-Score. These matrices were calculated for each of the QSE challenge categories. Precision (P_k) measures the accuracy of the positive predictions, indicating what proportion of instances classified as a certain category truly belong to that category. It is calculated as

$$P_k = \frac{TP_k}{TP_k + FP_k}, \tag{1}$$

where TP_k are the true positives and FP_k are the false positives for class k.

Recall (R_k), also known as sensitivity, measures the model's ability to identify all relevant instances of a class. It is calculated as

$$R_k = \frac{TP_k}{TP_k + FN_k}, \tag{2}$$

where FN_k are the false negatives for class k.

The F1-Score ($F1_k$) provides a single, balanced measure of a model's performance by calculating the harmonic mean of Precision and Recall. This is particularly useful for imbalanced datasets. The formula is

$$F1_k = 2 \times \frac{P_k \times R_k}{P_k + R_k}. \tag{3}$$

6 Results

This section presents the performance evaluation of TL-based and D&ML classifiers in categorizing QSE challenges using various experimental settings. As shown in Table 8, the TL models BERT and DistilBERT achieved an overall accuracy of 95% without any data augmentation, demonstrating their strong generalization capability in low-resource, domain-specific settings. Both models exhibited exceptional performance across all categories. BERT achieved a perfect F1-score of 0.99 in Theoretical and API Usage, while DistilBERT matched this level of performance in Learning, Tooling, and API Usage. These results confirm that pre-trained language models can effectively capture the semantic nuances of developer discussions in QSE without requiring synthetic data enhancement.

Compared to fine-tuned TL algorithms, the performance of DL models was highly dependent on data augmentation, as it can be seen in Table 8 that the performance of the fine-tuned CNN classifier is

Table 8 Performance of TL and DL classifiers in QSE (with and without data augmentation)

QSE concept	Classifier	Precision	Recall	F1-score	Data augmentation
Conceptual	BERT	0.99	0.94	0.97	No
	DistilBERT	0.80	0.99	0.89	No
	CNN	0.91	0.45	0.61	Yes
	CNN	0.37	0.36	0.36	No
	MLP	0.62	0.56	0.57	Yes
	MLP	0.39	0.54	0.45	No
Theoretical	BERT	0.99	0.99	0.99	No
	DistilBERT	0.99	0.78	0.88	No
	CNN	0.76	0.78	0.74	Yes
	CNN	0.35	0.40	0.38	No
	MLP	0.56	0.57	0.54	Yes
	MLP	0.42	0.23	0.30	No
Learning	BERT	0.89	0.99	0.89	No
	DistilBERT	0.99	0.99	0.99	No
	CNN	0.90	0.87	0.88	Yes
	CNN	0.36	0.26	0.30	No
	MLP	0.75	0.54	0.62	Yes
	MLP	0.20	0.23	0.25	No

Table 8 (Continued)

QSE concept	Classifier	Precision	Recall	F1-score	Data augmentation
Tooling	BERT	0.80	0.99	0.89	No
	DistilBERT	0.99	0.99	0.99	No
	CNN	0.88	0.91	0.89	Yes
	CNN	0.47	0.45	0.46	No
	MLP	0.64	0.68	0.65	Yes
	MLP	0.47	0.47	0.47	No
Errors	BERT	0.99	0.77	0.87	No
	DistilBERT	0.99	0.94	0.97	No
	CNN	0.93	0.95	0.94	Yes
	CNN	0.77	0.77	0.77	No
	MLP	0.81	0.87	0.84	Yes
	MLP	0.68	0.90	0.78	No
API usage	BERT	0.99	0.98	0.99	No
	DistilBERT	0.99	0.98	0.99	No
	CNN	0.93	0.80	0.86	Yes
	CNN	0.29	0.22	0.20	No
	MLP	0.59	0.44	0.49	Yes
	MLP	0.28	0.24	0.27	No
Overall accuracy					
BERT			95%		No
DistilBERT			95%		No
CNN			86%		Yes
CNN			51%		No
MLP			67%		Yes
MLP			54%		No

comparatively lower when exposed to actual developers' discussions on Q&A forums. The CNN model showed a significant improvement when trained with augmented data, achieving an overall accuracy of 86%, compared to only 51% without augmentation. This trend is consistent across categories: in Learning, CNN's F1-score increased from 0.30 to 0.88; in Errors, it rose from 0.77 to 0.94; and in API Usage, it jumped from 0.20 to 0.86. Similarly, the fine-tuned MLP model works well with augmentation, increasing its overall accuracy from 54% to 67%. Notably, in Errors, MLP improved its F1-score from 0.78 to 0.84, and in Tooling, from 0.47 to 0.65. We demonstrated the performances of CNN and MLP classifiers, while the detailed results of several D&ML algorithms are elaborated in our previously published paper [24].

These findings highlight a fundamental difference in model behavior: TL models like BERT and DistilBERT are effective out-of-the-box due to their extensive pre-training on large corpora, making them robust even with limited or imbalanced datasets. On the other hand, traditional DL models such as CNN and MLP rely heavily on data augmentation techniques, including random word swapping, deletion, and synonym insertion, to learn meaningful representations from sparse real-world forum data. Overall, while both TL and augmented DL approaches can achieve high performance, the superior results of TL models without augmentation make them a more practical, scalable, and efficient solution for the automated analysis of developer discussions in emerging domains, such as QSE. Moreover, we examined the baseline configurations of the top-performing classifiers, BERT and DistilBERT. As a central part of this analysis, we examined the Receiver Operating Characteristic (ROC) curves. ROC curves compare the true positive rate with the false positive rate to determine whether a classifier is sensitive or specific. By examining the accuracy metrics and the ROC curves, an analysis of the BERT model in organizing developer discussions

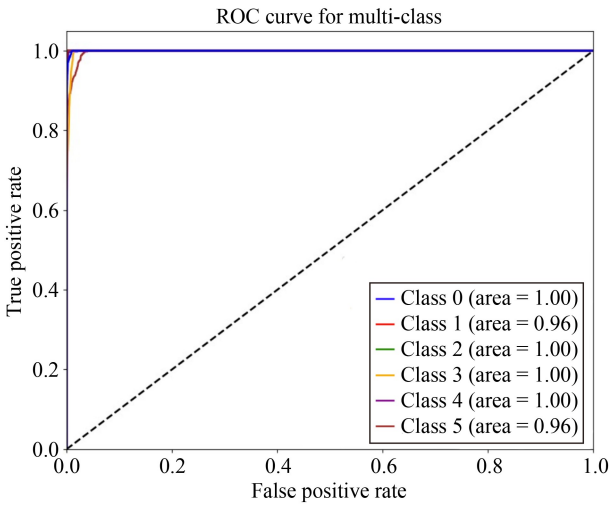


Fig. 6 ROC curves for BERT

from online distinct thematic categories is conducted. With AUC values near the ideal mark, as depicted in Fig. 6, each category demonstrated nearly perfect separation. This is evidenced by a high True Positive Rate (TPR) and a low False Positive Rate (FPR), with the curves approaching the upper-left corner of the plot. All three models achieved high macro-average AUC scores, indicating their consistent and precise ability to classify all categories accurately. These metrics highlight the proficiency of the BERT and DistilBERT models in accurately recognizing each discussion category while minimizing misclassifications, thereby confirming their robust performance in multiclass classification.

Furthermore, we compared the performances of the fine-tuned transformer (BERT) and deep learning (CNN) models by drawing their confusion matrices to identify false positives and negative rates. As shown in Figs. 7, 8, and 9, the fine-tuned BERT outperforms CNN classifiers both with and without augmentation, achieving low rates of false positives and negatives, resulting in more generalized classification results. In these confusion metrics, the row labels show the actual classes, while the column class labels show the predicted classes for each model. A confusion matrix is used to evaluate the performance of TL classifiers by summarizing the total number of correct and incorrect predictions, broken down by each classification class. In Fig. 7, the diagonal values, i.e., 168,143, etc., demonstrate the true positive values for each QSE category, such as tooling, conceptual, etc., showing how many developers' discussions in the Q&A forums are identified correctly. When comparing it to the diagonal values of CNN classifiers both with and without augmentation, as shown in Figs. 8 and 9, respectively. The fine-tuned BERT algorithm performed better and achieved better generalizability. In contrast, the off-diagonal values show false positives and false negatives values, in each confusion matrix, as shown in Figs. 7, 8, and 9, respectively. When compared, the fine-tuned BERT classifier shows comparatively lower rates of false positive and false negative values. These experimental findings demonstrate that fine-tuned transformer outperforms fine-tuned D&ML algorithms with relatively smaller and complex datasets, due

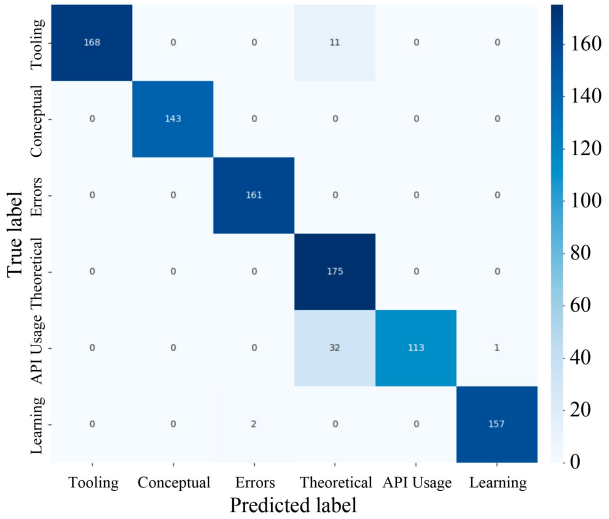


Fig. 7 Confusion matrix for BERT

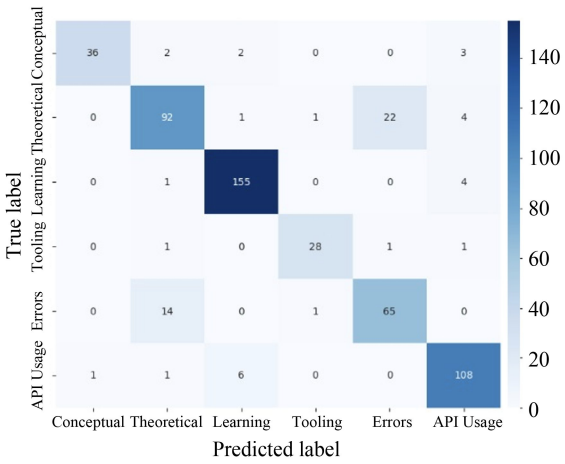


Fig. 8 Confusion matrix for CNN model with data augmentation

to its extensive training on large corpora and contextual understanding ability of encoders, which enables them to process complete sentences simultaneously, instead of word by word.

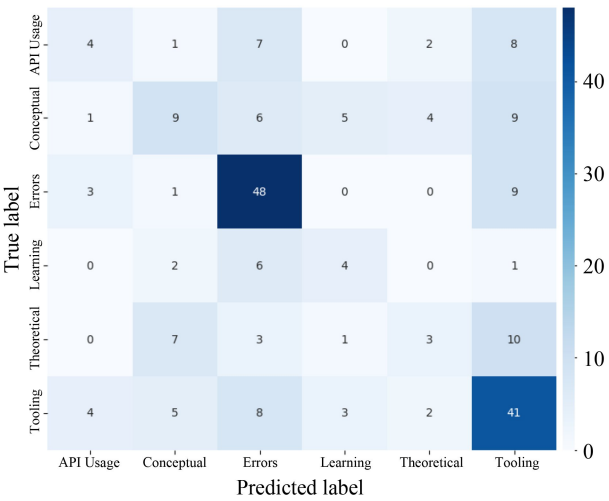


Fig. 9 Confusion matrix for CNN model without data augmentation

6.1 Global interpretability of the BERT model via SHAP analysis

To enhance transparency in the BERT classifier's decision-making process, we applied SHAP to identify the most influential textual features across all QSE challenge categories. The resulting SHAP summary plot, shown in Fig. 10, ranks the top features by their mean absolute SHAP value, the average impact on model output revealing which terms drive classification decisions. Each horizontal bar represents a key token such as "Programmer", "Register", or "Function", ordered by its overall importance, with color segments indicating the contribution of that feature to specific QSE categories: API Usage (blue), Conceptual (orange), Errors (green), Learning (red), Theoretical (purple), and Tooling (brown).

The highest-impact feature, "Programmer", exhibits a mean SHAP value of approximately 1.05, with dominant contributions from Learning (red) and Conceptual (orange), followed by Tooling (brown). This suggests that discussions involving programming roles frequently center on educational challenges or conceptual understanding, particularly in contexts where users require guidance or clarification. Similarly, "Register" has a strong influence (0.98), primarily linked to API Usage (blue) and Conceptual (orange), suggesting its relevance in discussions about hardware abstraction layers or low-level programming constructs. "Command" and "Function" also rank highly, with substantial contributions from Errors (green) and API Usage (blue), reflecting their frequent appearance in error messages or API invocation queries.

Notably, "Physics" shows a balanced mix of Conceptual (orange) and Theoretical (purple), highlighting the interplay between foundational theory and practical implementation in quantum computing discourse. Terms like "Circuit", "Quantum", and "Error" are strongly associated with Tooling (brown) and Errors (green), underscoring their role in debugging and simulation workflows. "Initial" and "Class" show significant contributions from API Usage (blue) and Tooling (brown), indicating their relevance in code initialization and object-oriented design contexts. Overall, this analysis demonstrates that the BERT model leverages both technical terminology and contextual semantics to make nuanced predictions, enabling developers to trace classification decisions back to specific linguistic cues and domain concepts.

6.2 Local interpretability via SHAP force plot

To further elucidate the model's decision-making process at the instance level, we present SHAP force plots for representative examples from each of the six QSE challenge categories. These visualizations break down individual predictions into contributions from specific input tokens, revealing how linguistic cues influence classification outcomes across various domains. As shown in Fig. 11(a), the model's high confidence in classifying the input as *Conceptual* (0.993892) is driven by key terms such as "example", "differ", "instantly", and "trying" highlighted with red color, which contributed positively in the prediction of Conceptual QSE challenge type. Moreover, "qubits", "circuit", and "understand" keywords contribute negatively to the prediction of the conceptual QSE challenge and force it onward to other QSE challenges. However, their contribution to the developer feedback is not high. Additionally, the phrase "how does that actually affect the computation?" further reinforces the abstract nature of the query, indicating a focus on fundamental behavior rather than implementation or debugging. This analysis demonstrates that the BERT model effectively identifies theoretical questions by recognizing language centered on quantum mechanics principles, distinguishing them from discussions related to learning, tooling, or errors.

Furthermore in Fig. 11(b), the model's high confidence in classifying the input as *Theoretical* (0.997788) is driven by key terms such as "NP-complete", "problems", "superposition", and "theoretical limitations", which reflect deep-level inquiry into

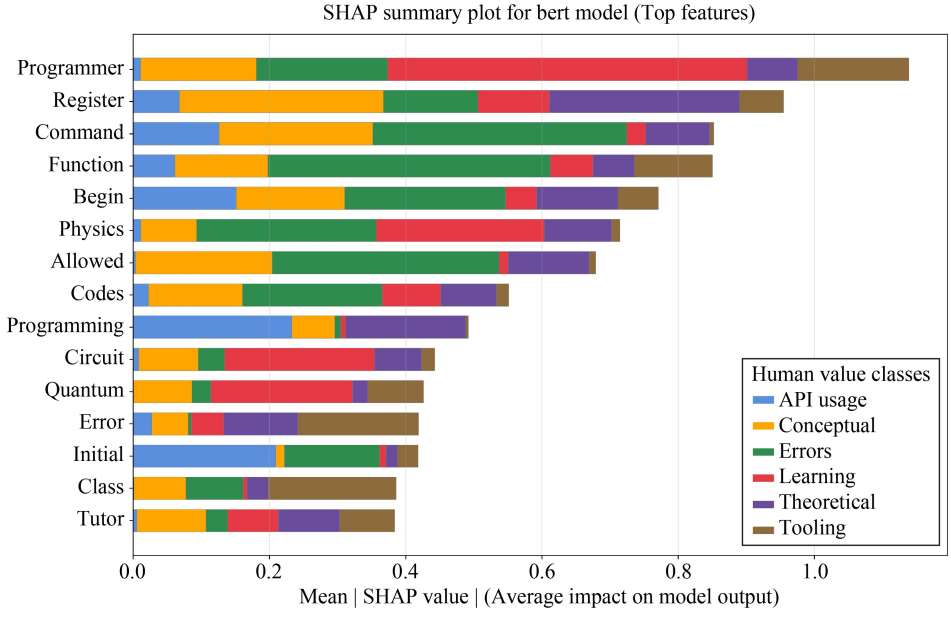


Fig. 10 SHAP summary plot for BERT model

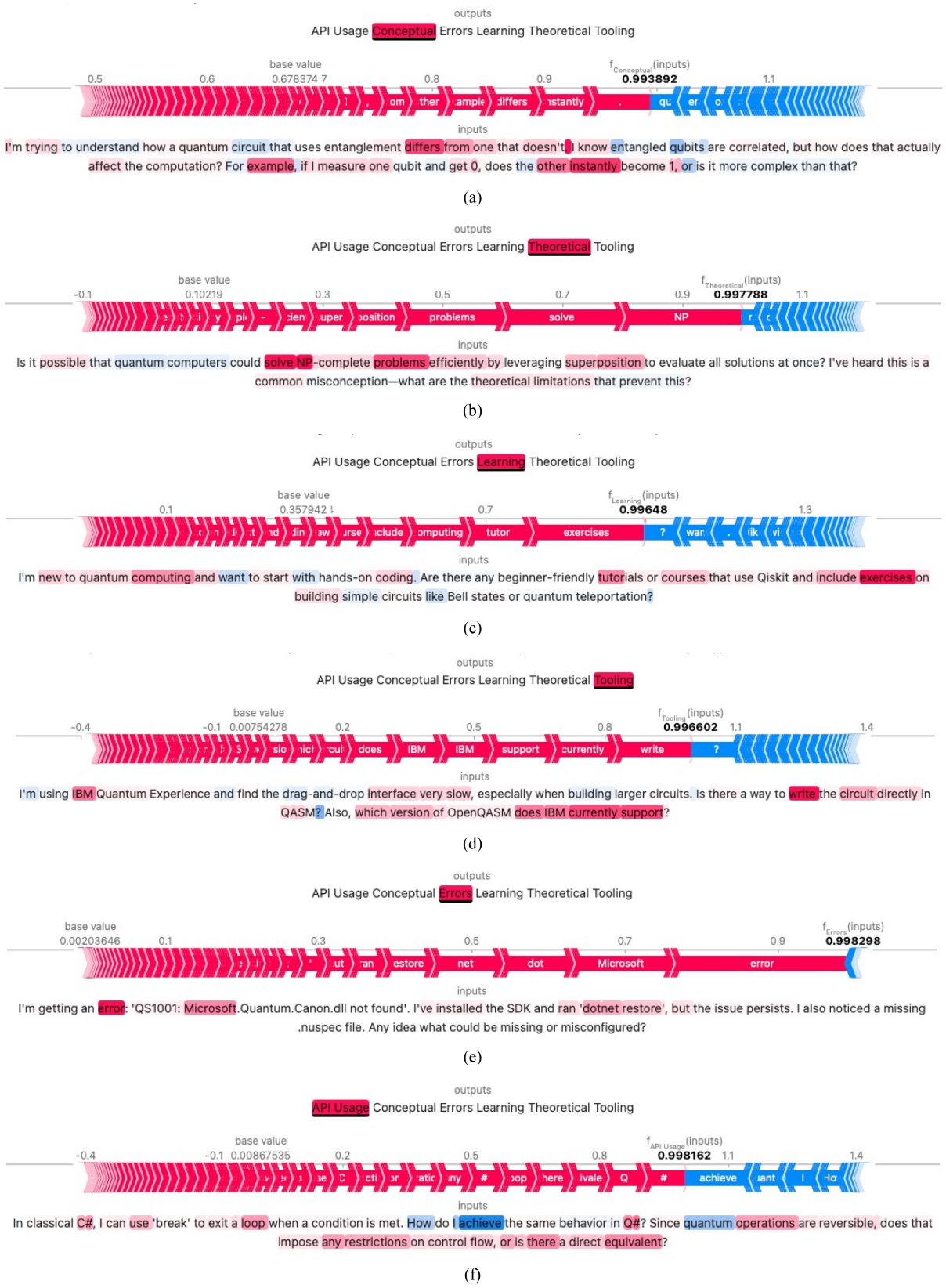


Fig. 11 SHAP Force Plots for BERT Model: (a) Conceptual, (b) Theoretical, (c) Learning, (d) Tooling, (e) Errors, and (f) API Usage

quantum computing's computational foundations. The phrase "solve NP-complete problems efficiently" signals a question about algorithmic complexity, while "leveraging superposition" highlights an understanding of quantum mechanics principles. The mention of "common misconception" further reinforces the abstract nature of the query, indicating that the user is engaging with foundational

knowledge rather than implementation or debugging. While the phrase "quantum computers" appear in blue adds a negative contribution and skews towards the other QSE challenges. However, their contribution is not extensive. This analysis demonstrates that the BERT model effectively distinguishes theoretical challenges by recognizing language centered on computational models, complexity theory, and scientific boundaries.

In addition, Fig. 11(c), the model's high confidence in classifying the input as *Learning* (0.99648) is driven by terms such as "tutorials", "courses", "exercises", and "computing", shown with red color (the darker, the more positive impact in the prediction) which collectively signal a request for educational resources. While "computing" appears in red, indicating a positive contribution, it is part of the phrase "quantum computing", a domain-specific term often associated with learning pathways. In contrast, "want" appears in blue, reflecting its contextual ambiguity; while commonly used in learning queries, it also appears in requests for tools or fixes, making it less discriminative. This analysis demonstrates that the BERT model distinguishes learning-related discussions by prioritizing domain-specific keywords over generic intent verbs, enabling accurate classification despite linguistic overlap across categories.

Notably, in Fig. 11(d), the model's high confidence in classifying the input as *Tooling* (0.996602) is driven by terms such as "IBM Quantum (with IBM more prominent)", "interface very slow", "QASM", "which version", and "write", which collectively signal a discussion about software tool usability and workflow efficiency. The phrase "interface very slow" indicates performance concerns, pointing to a possible tool. In contrast, words such as "?", "I'm" and "drag-and" contribute relatively negatively in the prediction of the tool QSE category, but with less intensity. This analysis demonstrates that the BERT model effectively identifies tooling challenges by recognizing language centered on software interfaces, performance limitations, and syntax-specific workflows.

For Fig. 11(e), the model's high confidence in classifying the input as *Errors* (0.998298) is driven by explicit technical indicators such as "error", "dotnetrestore", and "Microsoft", which collectively signal a software fault or dependency issue. The phrase "issue persists" reinforces the ongoing nature of the problem, while "dotnet restore" indicates an attempted resolution, confirming this as a troubleshooting scenario. Although "installed" and "SDK" appear in red, their context here supports error diagnosis rather than successful setup. This analysis demonstrates that the BERT model effectively identifies error-related discussions by recognizing language centered on specific error codes, missing files, and failed recovery attempts, distinguishing them from conceptual or learning queries.

Finally, in Fig. 11(f), the model's high confidence in classifying the input as *API Usage* (0.998162) is driven by technical terms such as "equivalent", "loop", "any restriction", "Q#", and "operations", which reflect a query about programming semantics in quantum languages. The words "how", "quantum", and "achieve", as shown in blue, contribute negatively to the prediction of API Usage. However, the intensity of these words is lower when deciding on classifying the developers' discussion into the more suitable QSE category. This analysis demonstrates that the BERT model effectively identifies API usage challenges by recognizing language centered on syntax, control structures, and functional equivalence between classical and quantum paradigms.

6.3 Comparative study with the state of the art

This section compares the proposed approach with existing methods in terms of scope, performance, and generalization. An overview of the comparison is shown in Table 9. Beyer et al. [23] use ML algorithms, including RF and SVM, to categorize Android-related discussions on SO. Their approach achieved an average accuracy of 90%, and it is limited to classifying developers' posts into Android-related challenges. In contrast, the proposed approach utilizes state-of-the-art transfer learning algorithms to contextually analyze developer discussions, achieving an overall accuracy of 95% and lower values of false positives and negatives. Yousuf and Sofi [55] proposed a rule-based approach to classify frequently occurring issues in quantum Qiskit repositories, achieving an accuracy of 85.2%. The proposed approach complements the El-Aoun et al. approach [13] by focusing on QSE from various Q&A platforms. However, we are classifying developers' discussions into frequently occurring challenges, whereas their approach identifies QSE challenges using topic modeling by analyzing developers' feedback. We argue that the proposed approach can help improve developers' experiences in quickly and better identifying related information on these Q&A forums by categorizing it based on the frequently occurring challenges, in addition to the existing tags. In addition,

Table 9 Comparative study with the state of the art

Reference papers	Method used	Highest precision, accuracy, recall	Dataset details	Data size
[23]	Random forest, SVM	Precision 90%, Accuracy 90%, Recall 90%	Android-related SO posts	1,000 posts
[55]	Rule-based classification	Accuracy 85.2%	Qiskit issues (36 repos)	12,910 issues (sampled 4,984)
[13]	Automated topic modeling	N/A	Stack Exchange forums, GitHub issues	1,755 issues/posts
[56]	Qualitative analysis	N/A	Stack Exchange forums, GitHub issues	385 discussions/issues
[57]	Mining analysis	N/A	21,000+ repositories	1.2 million+ commits
Our study	**BERT, RoBERTa, and DistilBERT**	**Accuracy 95%**	**Multiple Q&A forums (SO, QCSE, CSSE, AISE)**	**2,829 discussions**

Aktar et al. [56] conducted a qualitative analysis of discussions about quantum software development on Stack Exchange forums and GitHub issues. By examining 385 discussions/topics, they identified common challenges and solutions related to architecture decisions in quantum systems. The study categorized the decisions into implementation and technology-related issues, providing valuable insights into the linguistic patterns used by developers when discussing quantum software architectures. Furthermore, Upadhyay et al. [57] conducted a mining analysis on more than 21,000 quantum computing repositories, with over 1.2 million commits from more than 10,000 developers. The study aimed to uncover evolution and maintenance practices in quantum software development, highlighting key trends in programming languages, framework adoption, and developer growth. This large-scale analysis provides comprehensive insights into software engineering challenges and offers interesting insights into improving quantum development workflows, documentation, and tooling. In addition to our previously published work [24], this study demonstrates comparatively better precision, recall, F-measure, and overall accuracy by employing fine-tuned transformer algorithms, including BERT, RoBERTa, and DistilBERT, by contextually analyzing developers' discussions from various Q&A platforms and classifying them into frequently occurring QSE challenges with lower false negatives and positives rates. Overall, the proposed approach achieves 95% accuracy, outperforming our previous work [24] by approximately 6% (with data augmentation) and 41% without data augmentation. Additionally, the proposed approach enhances explainability by experimenting with SHAP, which identifies the frequent features used by the BERT classifier when classifying a particular developer discussion as a QSE challenge. Moreover, the proposed approach demonstrates why a particular developer's feedback is classified as a QSE challenge, along with its overall confidence rate. These insights can help quantum practitioners categorise Q&A posts based on the frequently identified challenges, providing easy access to related posts and solutions from other developers as an alternative option to the existing tags-based search.

■ 7 Discussion

The insights obtained from developer interactions on diverse social media platforms are powerful tools for software developers and vendors. Our experiments show that TL-based models outperform traditional D&ML baselines for classifying QSE-related developer discussions, with DistilBERT achieving the best overall performance on our dataset. In addition, SHAP-based analysis provides interpretable explanations by highlighting the terms and expressions most strongly associated with each QSE challenge category. These findings form the basis for the practical and research implications discussed below.

■ 7.1 Rationale for analyzing QSE-related developers' discussion

The recent interest, research findings, and success of quantum computing in various industries have paved the way for the development of quantum software. However, currently there is a lack of software development practices and methodologies for quantum software development [7]. Also, adopting classical software practices for quantum software development is challenging due to differences in control flow, debugging complexities, computation models, and resource limitations. To address these challenges, researchers started exploring Quantum Software Engineering (QSE) to develop and propose quantum software-specific development methodologies, tools, and practices. It is evident in the literature that software and quantum developers frequently refer to developers' Q&A forums to seek software solutions for development-related challenges [13,19]. For this purpose, we analyzed developers' discussions collected from various Q&A platforms to first annotate them with frequently occurring QSE challenges with humans and ChatGPT and then use this dataset to train a TL algorithm to automate the process. The proposed approach can help software developers quickly and accurately identify QSE-related information if the current Q&A interfaces are modified. For example, an additional menu can be added to display and categorize developers' discussions based on various QSE categories.

■ 7.2 Implications of research findings for software quantum developers and vendors

Software quantum developers can leverage research findings in several directions to improve the existing quantum development process and enhance their skills by contributing more effectively to the quantum life cycle. For example, since quantum errors (28%) have been identified as the most frequently occurring challenge using the content analysis approach, the quantum developers should focus on how to debug and troubleshoot quantum software. This also provides an opportunity for quantum vendors to develop tools that could help in diagnosing and identifying quantum errors in the code. Similarly, quantum developers can allocate time to get expertise in understanding basic quantum mechanisms, such as entanglement, quantum gates, and superposition, as conceptual challenges were reported the second most frequently in the Q&A platforms. Gaining expertise in this area will enable quantum developers to solve quantum problems more efficiently and enhance software design. These findings encourage software developers to arrange frequent hands-on workshops, tutorials, and lectures on quantum computing to familiarize developers with the emerging quantum concepts and development. Discussions on tooling inefficiencies highlight the gaps between resources and developer needs, suggesting opportunities for more intuitive tools, especially for quantum computing projects. Quantum developers should familiarize themselves with the limitations of existing tools and learn about best practices for implementing and using these tools to achieve improved outputs. At the same time, more fine-grained analysis should provide insights for the quantum vendors to identify areas where quantum developers have confronted challenges to develop more developer-friendly tools. Furthermore, API usage indicates that better documentation and standards can enhance productivity in the field. Quantum developers can take advantage of open source projects and community-driven examples to understand quantum APIs' pitfalls. Quantum vendors and researchers should work on improving the documentation and

toy examples to improve the usability of existing quantum APIs. Additionally, quantum developers should align with the latest research findings to enhance their theoretical understanding and identify ambiguous quantum areas. Quantum developers could take this opportunity to develop quantum-specific research and development communities to reduce the gap between the theoretical and practical implementation of quantum concepts. Finally, considering the growing interest and success of quantum computing, existing computer science, engineering, and software curricula should incorporate quantum-specific courses.

7.3 Implications of transformer-based learning for QSE

The superior performance of TL models, BERT, DistilBERT, and RoBERTa, over traditional DL architectures such as FNN, CNN, and RNN highlights a fundamental shift in how we approach text classification in emerging software engineering domains like QSE. Unlike DL models that rely on shallow feature extraction and local syntactic patterns, TL models leverage large-scale pre-training on general language corpora and fine-tuning on domain-specific data, enabling them to capture deep semantic relationships, contextual nuances, and long-range dependencies in developer discourse. This architectural advantage translates into significantly higher accuracy (95% vs. 86% for CNN (with data augmentation)) and more robust generalization across diverse challenge categories such as *Conceptual*, *Theoretical*, and *API Usage*. The improvements of TL algorithms become more prominent (95% vs. 51% for CNN (without data augmentation)) when compared to D&ML algorithms while processing developers' discussion without data augmentation. Additionally, fine-tuned transformer algorithms result in more reliable performances by minimizing the false positives and negatives rates.

A key reason for this performance gap lies in the nature of quantum programming discussions, which are often abstract, jargon-rich, and context-dependent. Terms like "entanglement", "superposition", or "Q#" carry specific meanings that vary subtly based on surrounding context—a nuance that DL models struggle to grasp due to limited contextual receptive fields. In contrast, TL models use self-attention mechanisms to dynamically weigh the importance of every word in a sentence, allowing them to distinguish between a query about theoretical complexity ("Can quantum computers solve NP-complete problems?") and one about practical implementation ("How do I write a loop in Q#?"). This sensitivity to semantic context makes TL models particularly well-suited for classifying unstructured, natural language content from forums where developers mix technical detail with personal experience and conceptual uncertainty.

Beyond accuracy, the success of TL models has broader implications for knowledge management and tool support in nascent software domains. Their ability to learn from relatively small, annotated datasets makes them viable even in low-resource settings like QSE, where labeled data is scarce. Furthermore, their compatibility with standard fine-tuning pipelines enables rapid adaptation to new platforms, APIs, or communities. As quantum software evolves, these models can be continuously retrained to reflect new challenges, making them ideal candidates for building intelligent assistants, automated documentation systems, and real-time support tools. The demonstrated effectiveness of TL models in this work sets a precedent for their adoption in other complex, language-intensive domains of software engineering, including AI/ML development, blockchain programming, and embedded systems.

7.4 Implications of SHAP for transformer-based models in QSE

The integration of SHAP with transformer-based models like BERT offers profound practical benefits for quantum vendors and developers. By making the model's decision-making transparent, SHAP enables developers and tool designers to trust and validate automated classification systems used to analyze forum discussions. For instance, when a post is classified as *API Usage* based on terms like "loop", "control flow", and "Q#", SHAP confirms that the decision is grounded in meaningful, contextually relevant programming constructs, not spurious correlations or noise. This level of justification is crucial for developing intelligent support tools, such as recommendation engines that suggest relevant documentation or code examples based on the detected challenge type. Moreover, project maintainers can use SHAP-generated insights to identify recurring pain points, such as confusion around quantum control structures, and prioritize improvements in language design, error messages, or API documentation. In educational contexts, instructors and curriculum designers can leverage these explanations to refine teaching materials by focusing on the most frequently misunderstood concepts, such as entanglement or superposition, as reflected in the influential terms surfaced by SHAP.

Beyond immediate usability, SHAP empowers researchers and practitioners to perform model-driven knowledge discovery in emerging domains where formal documentation is sparse. The ability to see which words drive predictions, like "QS1001" for *Errors* or "drag-and-drop interface" for *Tooling*, allows teams to detect evolving challenges and adapt their tooling proactively. For example, if SHAP consistently highlights dependency-related terms in error reports, it signals a need for better package management tooling in quantum SDKs. Similarly, the negative contribution of "classical C#" in an *API Usage* query reveals how developers frame their questions through contrast, suggesting that future tools should support comparative reasoning (e.g., "How is Q# different from C# for loops?"). For researchers, this interpretability layer transforms BERT from a classification engine into a *diagnostic instrument* for studying developer cognition, communication patterns, and learning trajectories. As quantum software matures, such explainable AI systems will not only support individual developers but also inform community-wide knowledge curation, making SHAP an essential component of any data-driven QSE research pipeline. These SHAP-derived high-impact keywords can be operationalized by tool vendors to (i) automatically tag and route incoming forum posts to the most relevant challenge category, (ii) improve search and recommendation by prioritizing challenge-indicative terms, and (iii) guide targeted documentation updates and troubleshooting

content aligned with the terms most associated with frequent error patterns.

7.5 Threats to validity

While processing and analyzing developer discussions, we encountered several potential threats to the validity of the proposed methodology and findings. Acknowledging these challenges is crucial to ensuring the credibility and reliability of the research outcomes.

7.5.1 Researcher bias

In the proposed study, the potential for researcher bias is significant, as the first, second, and third authors, along with an AI model such as ChatGPT, labeled and analyzed developer discussions to maintain neutrality and objectivity. However, our inherent attitudes and previous experiences subtly affected the analysis. This bias could skew the data's performance or emphasize specific findings. Acknowledging the presence of such biases is crucial to ensuring the integrity and reliability of our research findings. To mitigate this threat, we employed a human-AI collaborative annotation process, where disagreements between human coders and ChatGPT were resolved through negotiation and justification. This hybrid approach improved annotation consistency, achieving a Cohen's kappa of 0.46 indicating substantial agreement and reducing individual subjectivity in labeling.

7.5.2 Data preprocessing challenges

Data preprocessing is a critical step in the proposed approach, fraught with challenges, including addressing noisy data, removing biases, and resolving semantic opacity in developer discussions. These issues can significantly impact the quality and interpretability of data, potentially affecting subsequent analyses. Therefore, we must implement rich data preprocessing techniques to mitigate these issues, thereby enhancing the validity of our findings by ensuring that the data fed into our analytical models is as accurate and clean as possible.

7.5.3 Algorithm selection and fine tuning

The selection and fine-tuning of D&ML algorithms to examine developer discussions are fraught with several challenges. The proposed research's effectiveness and applicability depend heavily on selecting appropriate algorithms and optimizing them to handle the specific nuances of the QSE data. This process requires deep expertise and careful experimentation, as the selection of an imperfect algorithm or insufficient tuning can lead to misleading outcomes or decreased model effectiveness. We addressed this threat by conducting extensive comparative evaluation across multiple architectures, including FNN, CNN, RNN, and TL models, and using cross-validation to ensure stable performance estimation. Furthermore, the integration of SHAP provided post-hoc interpretability, allowing us to validate whether model decisions were driven by meaningful features rather than artifacts of overfitting or poor tuning.

7.5.4 Data quality concerns

Data quality is paramount for ensuring the reliability of the machine-learning models used in the study. High-quality data training and evaluation are essential for producing valid and generalizable results. Data noise, inherent biases, and semantic ambiguities within developer discussions must be addressed rigorously. These factors must be considered to maintain the trustworthiness of the research outputs, as the models may learn from flawed data, leading to unreliable predictions and insights. Our dataset was curated through a rigorous grounded theory and content analysis approach, resulting in 2,829 labeled instances. Conflicts in initial annotations were systematically resolved using ChatGPT-generated rationales, leading to a conflict-free final dataset. This iterative validation process significantly enhanced data quality and reduced the risk of propagating labeling errors into model training.

7.5.5 Ever-evolving landscape of online forums

The vibrant and ever-changing landscape of online forums and developer communities presents additional challenges for maintaining the scalability and generalizability of the proposed approach. These platforms have evolved rapidly with new models and continually emerging topics. Thus, the proposed methodology requires ongoing adjustments and updates to remain relevant and effective in capturing the current state of developer discussions across various forums. To address this, the proposed framework is designed to be extensible, supporting periodic retraining with newly collected data. Moreover, the use of pre-trained language models like BERT improves adaptability, as they can be efficiently fine-tuned on updated corpora. Future work will explore automated drift detection mechanisms to trigger model updates when shifts in discussion patterns are identified.

7.5.6 External validity and generalization

Our dataset is derived from Q&A platforms (e.g., Stack Overflow and Stack Exchange communities), which may not fully represent challenges discussed in industrial settings or other communication channels (e.g., issue trackers or team chats). Therefore, the observed challenge distribution and model performance may be influenced by platform-specific participation and tagging practices. In addition, the dataset is imbalanced across challenge categories, which can affect classifier generalization to under-represented classes. To mitigate this, we apply sampling strategies during training and report per-class performance. As future work, we plan to validate the proposed approach on larger and more diverse data sources and to investigate alternative imbalance-handling techniques (e.g., SMOTE-based variants and cost-sensitive learning) to further assess robustness and generalizability.

8 Conclusions and future work

This study presents a TL-based approach to classifying developer challenges in QSE-related frequently occurring challenges, achieving 95% accuracy through fine-tuned TL-based models, BERT, and DistilBERT, outperforming traditional D&ML methods. By analyzing discussions from multiple Q&A forums and employing SHAP for explainability, the model not only delivers high performance, but also provides interpretable insights into feature

contributions, revealing key linguistic patterns associated with challenge categories such as Conceptual, Learning, Error, API Usage, Theoretical, and Tooling. The integration of SHAP provides both global and local interpretability, revealing how specific terms influence model predictions. This work advances the automation of knowledge extraction in QSE, offering a scalable, transparent framework to support the growing quantum development community. As future work, we plan to conduct empirical evaluation with quantum practitioners to further validate the taxonomy and assess the practical usefulness of the proposed classification approach in real-world settings.

Furthermore, future studies will focus on key areas to expand and enhance the capabilities of the automated categorization model. We plan to explore other forums and discussion platforms popular in the QSE community to collect more data. We aim to provide a more intuitive interface and toolchain that classifies developers' discussions into frequently occurring challenges on the run. These efforts aim to develop robust collaborative relationships with field professionals, maintaining the model's relevance and effectiveness in addressing practical barriers in QSE. Moreover, we plan to extend the proposed approach to other emerging technologies' discussions on Q&A forums, such as blockchain, large language models, deep learning, and transfer learning. Additionally, we aim to develop a toolset for the proposed approach by implementing the best-performing D&ML classifiers and demonstrating their applicability for quantum software developers.

Acknowledgements

This work was supported by the following projects: Securing the Quantum Software Stack (SeQuSoS)(24304955), funded by Business Finland; and Classi|Q⟩(24304728121) , funded jointly by the City of Oulu and the University of Oulu, Finland.

Competing interests

The authors declare that they have no competing interests or financial conflicts to disclose.

References

[1] Nielsen M A, Chuang I L. Quantum Computation and Quantum Information. Cambridge: Cambridge University Press, 2010

[2] Preskill J. Reliable quantum computers. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 1998, 454(1969): 385−410

[3] Arute F, Arya K, Babbush R, Bacon D, Bardin J C, Barends R, et al. Quantum supremacy using a programmable superconducting processor. Nature, 2019, 574(7779): 505−510

[4] Gambetta J. IBM'S roadmap for scaling quantum technology. Yorktown Heights: IBM Quantum, 2020 (https://www.ibm.com/quantum/blog/ibm-quantum-roadmap)

[5] Rigetti Computing. Create New Opportunities with Tomorrow's Most Advanced Computing Technology. Berkeley: Rigetti Computing, 2020 (https://www.rigetti.com/why-quantum)

[6] Piattini M, Peterssen G, Pérez-Castillo R, Hevia J L, Serrano M A, et al. The Talavera manifesto for quantum software engineering and programming. In: Proceedings of the 1st International Workshop on the Quantum Software Engineering & Programming. 2020, 1−5

[7] Murillo J M, Garcia-Alonso J, Moguel E, Barzen J, Leymann F, Ali S, Yue T, Arcaini P, Pérez-Castillo R, García-Rodríguez de Guzmán I, Piattini M, Ruiz-Cortés A, Brogi A, Zhao J, Miranskyy A, Wimmer M. Quantum software engineering: roadmap and challenges ahead. ACM Transactions on Software Engineering and Methodology, 2025, 34(5): 154

[8] Mandal A K, Nadim M, Roy C K, Roy B, Schneider K A. Quantum software engineering and potential of quantum computing in software engineering research: a review. Automated Software Engineering, 2025, 32(1): 27

[9] Zhao J. Quantum software engineering: landscapes and horizons. 2021, arXiv preprint arXiv: 2007.07047

[10] Svore K, Geller A, Troyer M, Azariah J, Granade C, Heim B, Kliuchnikov V, Mykhailova M, Paz A, Roetteler M. Q#: enabling scalable quantum computing and development with a high-level DSL. In: Proceedings of Real World Domain Specific Languages Workshop 2018. 2018, 7

[11] JavadiAbhari A, Patil S, Kudrow D, Heckey J, Lvov A, Chong F T, Martonosi M. ScaffCC: scalable compilation and analysis of quantum programs. Parallel Computing, 2015, 45: 2−17

[12] IBM Quantum. Qiskit: an open-source SDK for working with quantum computers. IBM, 2024 (https://www.ibm.com/quantum/qiskit)

[13] El Aoun M R, Li H, Khomh F, Openja M. Understanding quantum software engineering challenges an empirical study on stack exchange forums and GitHub issues. In: Proceedings of 2021 IEEE International Conference on Software Maintenance and Evolution. 2021, 343−354

[14] Yue T, Ali S, Arcaini P. Towards quantum software requirements engineering. In: Proceedings of 2023 IEEE International Conference on Quantum Computing and Engineering. 2023, 161−164

[15] Ahmad A, Khan A A, Waseem M, Fahmideh M, Mikkonen T. Towards process centered architecting for quantum software systems. In: Proceedings of 2022 IEEE International Conference on Quantum Software. 2022, 26−31

[16] Khan A A, Ye B, Akbar M A, Khan J A, Mougouei D, Ma X. Mining Q&A platforms for empirical evidence on quantum software programming. 2025, arXiv preprint arXiv: 2503.05240

[17] Akbar M A, Khan A A, Rafi S. A systematic decision-making framework for tackling quantum software engineering challenges. Automated Software Engineering, 2023, 30(2): 22

[18] Ali S, Yue T. Quantum software testing: a brief introduction. In: Proceedings of 2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings. 2023, 332−333

[19] Ahmad A, Feng C, Ge S, Yousif A. A survey on mining stack overflow: question and answering (Q&A) community. Data Technologies and Applications, 2018, 52(2): 190−247

[20] De Dieu M J, Liang P, Shahin M. How do OSS developers reuse architectural solutions from Q&A sites: an empirical study. IEEE Transactions on Software Engineering, 2025, 51(7): 2015−2043

[21] Khan A A, Khan J A, Akbar M A, Zhou P, Fahmideh M. Insights into software development approaches: mining Q &A repositories.

Empirical Software Engineering, 2024, 29(1): 8

[22] Li T, Zhang X, Wang Y, Zhou Q, Wang Y, Dong F. Machine learning for requirements engineering (ML4RE): a systematic literature review complemented by practitioners' voices from stack overflow. Information and Software Technology, 2024, 172: 107477

[23] Beyer S, Macho C, Di Penta M, Pinzger M. What kind of questions do developers ask on stack overflow? A comparison of automated approaches to classify posts into question categories. Empirical Software Engineering, 2020, 25(3): 2258−2301

[24] Husain M, Khan M S, Khan J A, Khan N D, Khan A, Akbar M A. Exploring developers discussion forums for quantum software engineering: a fine-grained classification approach using large language model (ChatGPT). In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. 2025, 1742−1755

[25] Gill S S, Kumar A, Singh H, Singh M, Kaur K, Usman M, Buyya R. Quantum computing: a taxonomy, systematic review and future directions. Software: Practice and Experience, 2022, 52(1): 66−114

[26] Vietz D, Barzen J, Leymann F, Wild K. On decision support for quantum application developers: categorization, comparison, and analysis of existing technologies. In: Proceedings of the 21st International Conference on Computational Science. 2021, 127−141

[27] Haghparast M, Mikkonen T, Nurminen J K, Stirbu V. Quantum software engineering challenges from developers' perspective: mapping research challenges to the proposed workflow model. In: Proceedings of 2023 IEEE International Conference on Quantum Computing and Engineering. 2023, 173−176

[28] Treude C, Barzilay O, Storey M A. How do programmers ask and answer questions on the web? (NIER track). In: Proceedings of the 33rd International Conference on Software Engineering. 2011, 804−807

[29] Iftikhar H U, Rehman A U, Kalugina O A, Umer Q, Khan H A. Deep learning-based correct answer prediction for developer forums. IEEE Access, 2021, 9: 128166−128177

[30] Zhu W, Zhang H, Hassan A E, Godfrey M W. An empirical study of question discussions on stack overflow. Empirical Software Engineering, 2022, 27(6): 148

[31] Khan J A, Yasin A, Fatima R, Vasan D, Khan A A, Khan A W. Valuating requirements arguments in the online user's forum for requirements decision-making: the crowdRE-VArg framework. Software: Practice and Experience, 2022, 52(12): 2537−2573

[32] Ali Khan J, Liu L, Wen L. Requirements knowledge acquisition from online user forums. IET Software, 2020, 14(3): 242−253

[33] Beyer S, Pinzger M. A manual categorization of android app development issues on stack overflow. In: Proceedings of 2014 IEEE International Conference on Software Maintenance and Evolution. 2014, 531−535

[34] Zhang H, Shafiq M O. Survey of transformers and towards ensemble learning using transformers for natural language processing. Journal of Big Data, 2024, 11(1): 25

[35] de Araújo A F, Marcacini R M. RE-BERT: automatic extraction of software requirements from app reviews using BERT language model. In: Proceedings of the 36th Annual ACM Symposium on Applied Computing. 2021, 1321−1327

[36] Sainani A, Anish P R, Joshi V, Ghaisas S. Extracting and classifying requirements from software engineering contracts. In: Proceedings of the 28th IEEE International Requirements Engineering Conference. 2020, 147−157

[37] Deshpande G, Sheikhi B, Chakka S, Zotegouon D L, Masahati M N, Ruhe G. Is BERT the new silver bullet?-An empirical investigation of requirements dependency classification. In: Proceedings of the 29th IEEE International Requirements Engineering Conference Workshops. 2021, 136−145

[38] Hey T, Keim J, Koziolek A, Tichy W F. NorBERT: transfer learning for requirements classification. In: Proceedings of the 28th IEEE International Requirements Engineering Conference. 2020, 169−179

[39] Henao P R, Fischbach J, Spies D, Frattini J, Vogelsang A. Transfer learning for mining feature requests and bug reports from tweets and app store reviews. In: Proceedings of the 29th IEEE International Requirements Engineering Conference Workshops. 2021, 80−86

[40] Kaur K, Kaur P. BERT-RCNN: an automatic classification of app reviews using transfer learning based RCNN deep model. 2023 (https://www.researchsquare.com/article/rs-2503700/v1)

[41] Bilal M, Almazroi A A. Effectiveness of fine-tuned BERT model in classification of helpful and unhelpful online customer reviews. Electronic Commerce Research, 2023, 23(4): 2737−2757

[42] Devlin J, Chang M W, Lee K, Toutanova K. BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2019, 4171–4186

[43] Strauss A L, Corbin J M. Basics of Qualitative Research. 2nd ed. Thousand Oaks: Sage Publications, 1998

[44] Neuendorf K A. The Content Analysis Guidebook. 2nd ed. Los Angeles: SAGE, 2017

[45] Khan N D, Khan J A, Li J, Ullah T, Zhao Q. Mining software insights: uncovering the frequently occurring issues in low-rating software applications. PeerJ Computer Science, 2024, 10: e2115

[46] Khan N D, Khan J A, Li J, Ullah T, Zhao Q. Leveraging large language model ChatGPT for enhanced understanding of end-user emotions in social media feedbacks. Expert Systems with Applications, 2025, 261: 125524

[47] Rosen C, Shihab E. What are mobile developers asking about? A large scale study using stack overflow. Empirical Software Engineering, 2016, 21(3): 1192−1223

[48] Allamanis M, Sutton C. Why, when, and what: analyzing stack overflow questions by topic, type, and code. In: Proceedings of the 10th Working Conference on Mining Software Repositories. 2013, 53−56

[49] Uddin G, Khomh F. Automatic mining of opinions expressed about APIs in stack overflow. IEEE Transactions on Software Engineering, 2021, 47(3): 522−559

[50] Baquero J F, Camargo J E, Restrepo-Calle F, Aponte J H, González F A. Predicting the programming language: extracting knowledge from stack overflow posts. In: Proceedings of the 12th Colombian Conference on Computing. 2017, 199−210

[51] Tan Y, Xu S, Wang Z, Zhang T, Xu Z, Luo X. Bug severity prediction using question-and-answer pairs from stack overflow. Journal

of Systems and Software, 2020, 165: 110567

[52] González-Carvajal S, Garrido-Merchán E C. Comparing BERT against traditional machine learning text classification. 2021, arXiv preprint arXiv: 2005.13012

[53] Sanh V, Debut L, Chaumond J, Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. 2020, arXiv preprint arXiv: 1910.01108

[54] Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V. RoBERTa: a robustly optimized BERT preprint approach. 2019, arXiv preprint arXiv: 1907.11692

[55] Yousuf M M, Sofi S A. Bug classification in quantum software: a rule-based framework and its evaluation. Automated Software Engineering, 2026, 33(1): 39

[56] Aktar M S, Liang P, Waseem M, Tahir A, Ahmad A, Zhang B, Li Z. Architecture decisions in quantum software systems: an empirical study on stack exchange and GitHub. Information and Software Technology, 2025, 177: 107587

[57] Upadhyay K, Chhetri V, Siddique A B, Farooq U. Analyzing the evolution and maintenance of quantum software repositories. In: Proceedings of 2025 IEEE International Conference on Quantum Software. 2025, 173−184



Nek Dil KHAN received the BSc degree in software engineering from the University of Science and Technology Bannu, Pakistan, and the MS degree in software engineering from Northeastern University, China. He is currently pursuing the PhD degree in software engineering at Beijing University of Technology (BJUT), China. His areas of research interest include software engineering, requirements engineering, CrowdRE, mining software repositories, human values in Software, quantum software engineering, empirical software engineering, NLP, health informatics, large language models (LLMs), transfer learning (TL), and explainable AI. He has published more than ten research articles in well-reputed journals and international conferences.



Javed Ali KHAN received the PhD degree in software engineering from Tsinghua University, China. He is currently working as a Senior Lecturer with the Foundation of Software Engineering (FSE) Group, Department of Computer Science, University of Hertfordshire, UK, teaching core software engineering courses. Previously, he worked as the assistant professor cum Chairperson with the Department of Software Engineering, University of Science and Technology Bannu, Pakistan. His areas of research interest include software engineering, requirements engineering, CrowdRE, mining software repositories, human values in software, quantum software engineering, empirical software engineering, NLP, and health informatics. Javed has published more than 90 peer-reviewed conference and journal papers.



Mobashir HUSAIN received his BSc degree in software engineering from the University of Science and Technology Bannu (UST Bannu), Pakistan in 2021, and his MS degree in computer software engineering from the University of Engineering and Technology Mardan, Pakistan in 2024. He worked as a lab engineer at UST Bannu and as an IT officer at Alkhidmat Hospital, Pakistan. Since September 2025, he has been a lecturer at Sarhad University of Science and Information Technology (SUIT), Pakistan. His research interests include software engineering, quantum software engineering, AI, software requirements engineering, human-computer interaction, software systems management, software analytics, NLP, big data, sentiment mining, and AI-driven software engineering.



Muhammad Sohail KHAN is an associate professor at the Department of Computer Software Engineering, University of Engineering and Technology Mardan, Pakistan. He received his MS degree from Computer Software Engineering Department, University of Engineering and Technology Peshawar, Pakistan in 2012 and completed his PhD degree from JeJu National University, Republic of Korea in 2016. He had been a part of the software development industry in Pakistan as a designer and developer. The major focus of his work is the investigation and application of alternate programming strategies to enable the involvement of masses in Internet-of-Things application design and development. His research interests include IoT, Blockchain, Cloud computing, Machine Learning, End-User Programming, Human-Computer Interactions and Empirical Software Engineering.



Arif Ali KHAN received the PhD degree in software engineering from City University of Hong Kong, China. He is currently working as an associate professor in the M3S Empirical Software Engineering Research Unit at the University of Oulu, Finland. Previously, he held academic positions at the University of Jyväskylä, Finland and Nanjing University of Aeronautics and Astronautics, China. His research interests include quantum software engineering, empirical software engineering, software process improvement, and evidence-based software engineering. He has published more than 100 peer-reviewed articles and serves as an editor and reviewer for leading software engineering venues.



Muhammad Azeem AKBAR is an associate professor and adjunct professor (docent) in software process improvement and management at Lappeenranta University of Technology (LUT University), Finland. He received his PhD degree in Software Engineering from Chongqing University, China in 2019. He previously held postdoctoral and research positions at the Nanjing University of Aeronautics and Astronautics, China and Lero–The Irish Software Research Centre, Ireland. Since joining LUT University in 2021, he has been recognized with multiple awards for excellence in teaching and academic advising. He has published over 130 research articles in reputed journals and conferences. His multidisciplinary research integrates software engineering with Large Language Models (LLMs), data analytics, and artificial intelligence to address real-world challenges in modern software systems.



Shahid HUSSAIN is an assistant professor in the Department of Computer Science and Software Engineering at Penn State University Behrend, USA. Previously, he served as a Pro tem faculty member at the University of Oregon, USA and an assistant professor at COMSATS University, Islamabad. He has held leadership and academic roles at Namal College (University of Bradford affiliate), UK and the University of Peshawar, Pakistan. His research focuses on software engineering practices for high-performance computing (HPC), automated documentation for open-source software, and software fairness analysis. He also explores the implications of machine learning and AI in software design. He is a long-standing research collaborator with international teams including SERC, MID Lab, and SE-NUAA, contributing to global software engineering research since 2014.